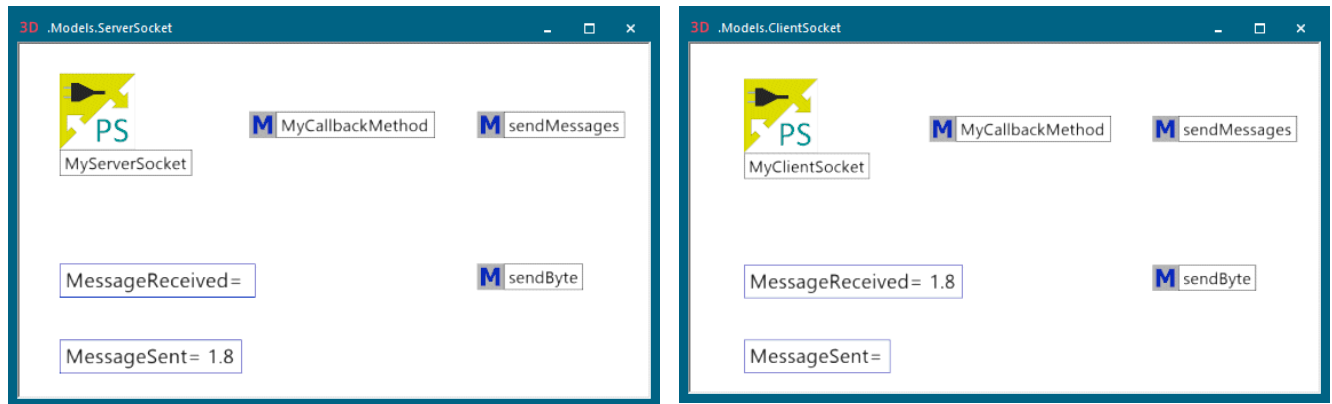
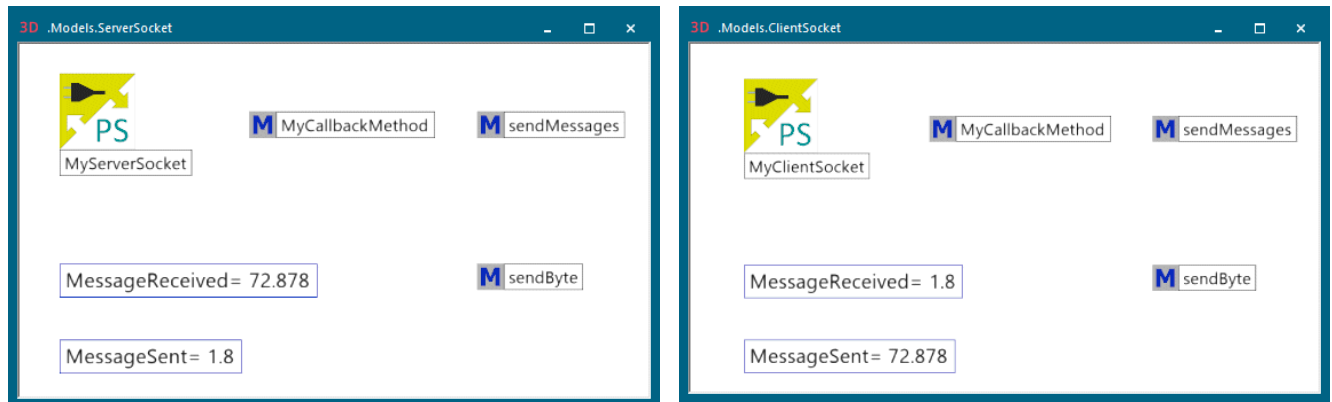




- Then, click the method *sendMessages* in the *Frame ClientSocket* with the right mouse button and select **Run** on the context menu. Watch what the variables show. *MessageSent* in the *Frame ClientSocket* shows that the method calculated the number **72.878** and sent it to the variable *MessageReceived* in the *Frame ServerSocket*, which naturally also shows the number **72.878**. Also check what the *Console* shows.



Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.

Back to [Import Data for the Simulation](#)

Setting Parameters in the Model

Set Parameters in the Model

Introduces you on how to enter parameters into your simulation model and execute simulation runs with these parameters.

You can:

- [Set Parameters for Objects in Your Own Dialog](#)

- [Set Parameters with the AttributeExplorer](#)

Compare [Model Random Processes](#)

Compare [Run Simulation Experiments](#)

Setting Parameters for Objects in Your Own Dialog

Set Parameters for Objects in Your Own Dialog

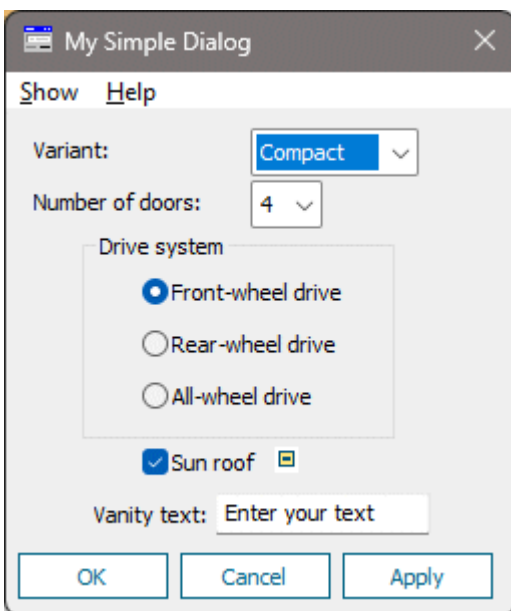
If you want to get information from the user of your simulation model, be it you yourself, a colleague in another department or one of your suppliers, you can create dialog boxes, which look like the built-in *Plant Simulation* dialog boxes and which work in the same way, too.

With the object [Dialog](#)  you can:

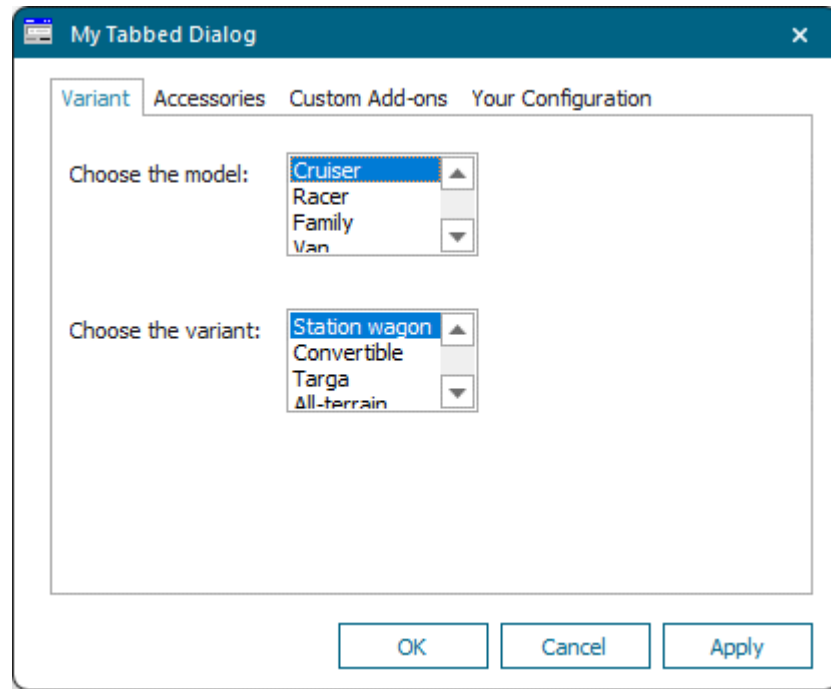
- Provide the user of your simulation model with a simple user interface for complex simulation models. This way you can ask the user to select or to enter information, which *Plant Simulation* needs to carry out a task.
- Prevent the user from manipulating a *Frame* in which you modeled a complex machine, etc. To accomplish this, enter a *Method* object as an **Open Control** for the *Frame* and insert a user-defined dialog with simulation settings into the *Frame*. When the user double-clicks the *Frame*, *Plant Simulation* will not open the *Frame* as such, but call the *Method* whose name you entered. The *Method* in turn executes the **Open Control**, i.e., it opens a user-defined dialog, where the user can then select the settings for the simulation run.

Note

The **display window** of the object *Dialog* that contains menus does not use the standard Siemens PLM theme, but your Windows theme.



The **display window** of the object *Dialog* that **does not contain menus** uses the standard Siemens PLM theme.



You can insert the *Dialog* into your simulation model from the folder **UserInterface** in the *Class Library* or from the toolbar **User Interface** in the *Toolbox*.



Each *Dialog* object manages a single dialog box. If you need more than one user-defined dialog boxes in your simulation model, insert as many *Dialog* objects as you need *Dialogs*.

You will:

- [Plan Layout and Structure of Your Dialog](#)
- [Design a Simple Dialog](#)
- [Design a Tabbed Dialog](#)
- [Program Actions which the Dialog Items Execute](#)
- [Program Actions for Interacting with the Dialog](#)
- [Create a Dialog Dynamically](#)

In which order you go about creating the user-defined dialog is up to you and depends on your personal work habits. You can first design the layout and then do your programming. You have to program the actions, which the individual dialog items execute, and the actions, which *Plant Simulation* executes, when the user opens the user-defined dialog, when he clicks **Apply**, and when he closes it. Or you can start programming and then design the layout. Or you can mix both techniques to achieve the best result.

Back to Set Parameters in the Model

Plan Layout and Structure of Your Dialog

Before you start designing your dialog, take some time to think about its layout. Orient the dialog items in the direction the user reads, in Western countries this is from left-to-right and from top-to-bottom.

Place the primary dialog item with which the user interacts as close to the upper corner of the dialog as possible. Do the same when orienting dialog items within groups in your dialog. Place the major command buttons stacked along the right border of the dialog or lined up across the bottom of the dialog. Place the most important button as the first button in a set of buttons.

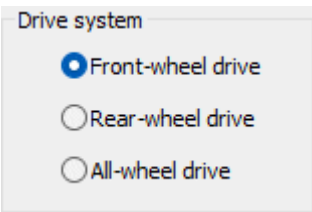
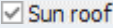
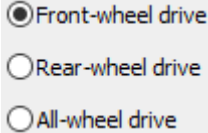
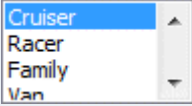
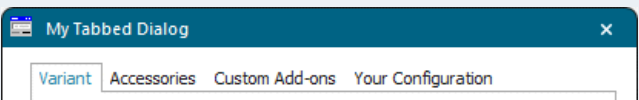
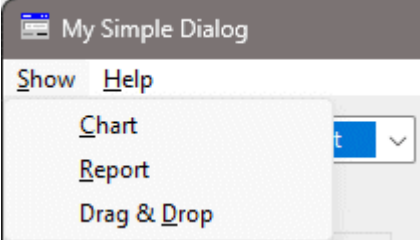
Also, take some time to decide about the structure of your dialog. Ask yourself which dialog items you do need, and if you can show them all in a **simple dialog** with a flat hierarchy in the dialog or if you need to present the information on **tabs**.

The structure you decide upon determines how and in which order you create the dialog items. Let us say, you want to show the dialog items on tabs. Then, you first have to create the **Tab Control**, which is the container that accommodates the individual tabs. After this you have to create the individual tabs, and finally you can insert the dialog items onto each one of these **tabs**.

The same applies to **groups** of dialog items. First, create the group and then insert and position the items you want to add to that group. The y-coordinate you enter for items in a group relate to the y-coordinate within that group, not within the overall dialog.

For both dialog types you can create menus and you can create these types of dialog items.

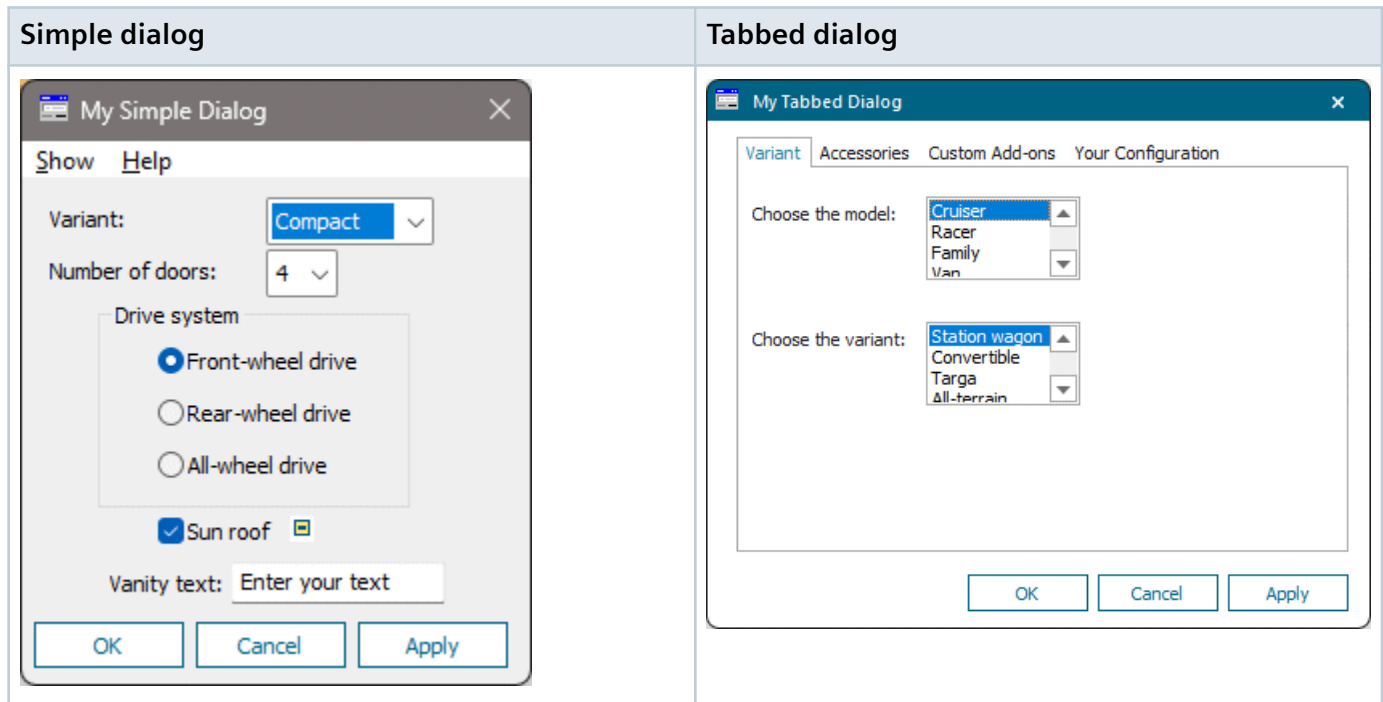
Dialog item	Icon	Looks like this
Static Text Box		
Text Box		
Button		
Drop-Down List Box		

Dialog item	Icon	Looks like this
Group Box		
Check Box		
Radio Button		
List Box		
Image		
List View		
Tab Control		
Tab Page		
Menu/menu command		

Depending on your modeling needs, you will:

- [Design a Simple Dialog](#)
- [Design a Tabbed Dialog](#)

For both types you have to program the actions, which the dialog items execute, when the user selects them.



Note

The **display window** of the object *Dialog* that contains menus does not use the standard Siemens PLM theme, but your Windows theme.

The **display window** of the object *Dialog* that does not contain menus uses the standard Siemens PLM theme.

Note

Plant Simulation shows the dialog with the settings you select under **Personalization > Font** as well as with the settings you select under **Start > Settings > System > Display > Scale & layout**. When you change these settings, the dialog scales text, while the images always retain their size in pixels. For this reason it can happen that text overlaps the image. We recommend to test your dialogs with different display settings to make sure that everything works as you expect it.

Compare [Program Actions which the Dialog Items Execute](#)

Compare [Program Actions for Interacting with the Dialog](#)

Compare [Create a Dialog Dynamically](#)

[Back to Set Parameters for Objects in Your Own Dialog](#)

Design a Simple Dialog

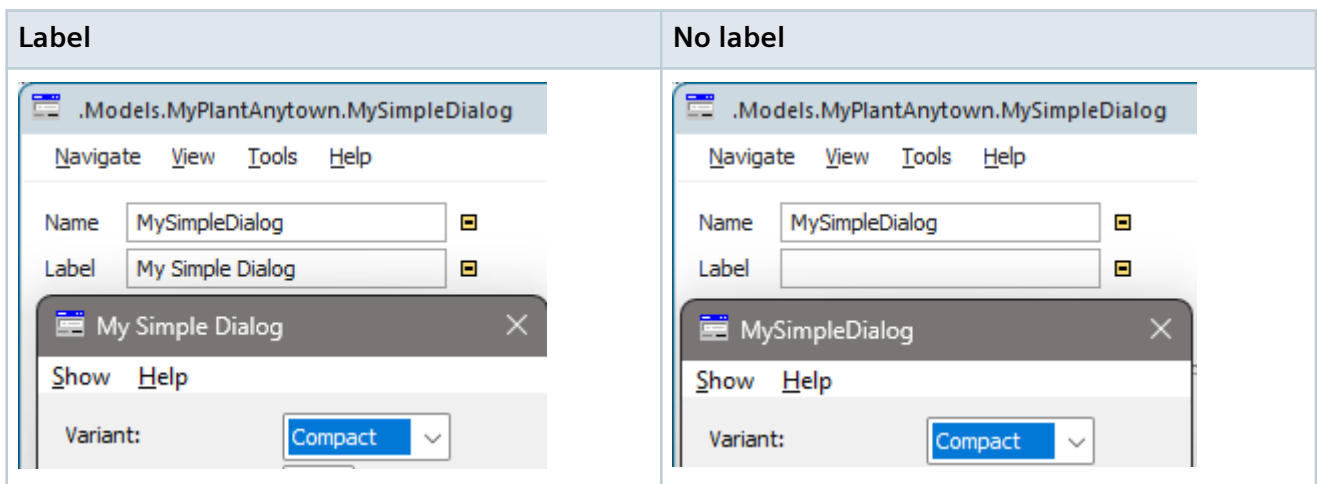
Design a Simple Dialog

If you only need the user to select or to enter a small number of items, you can present them directly in the dialog in a flat structure.

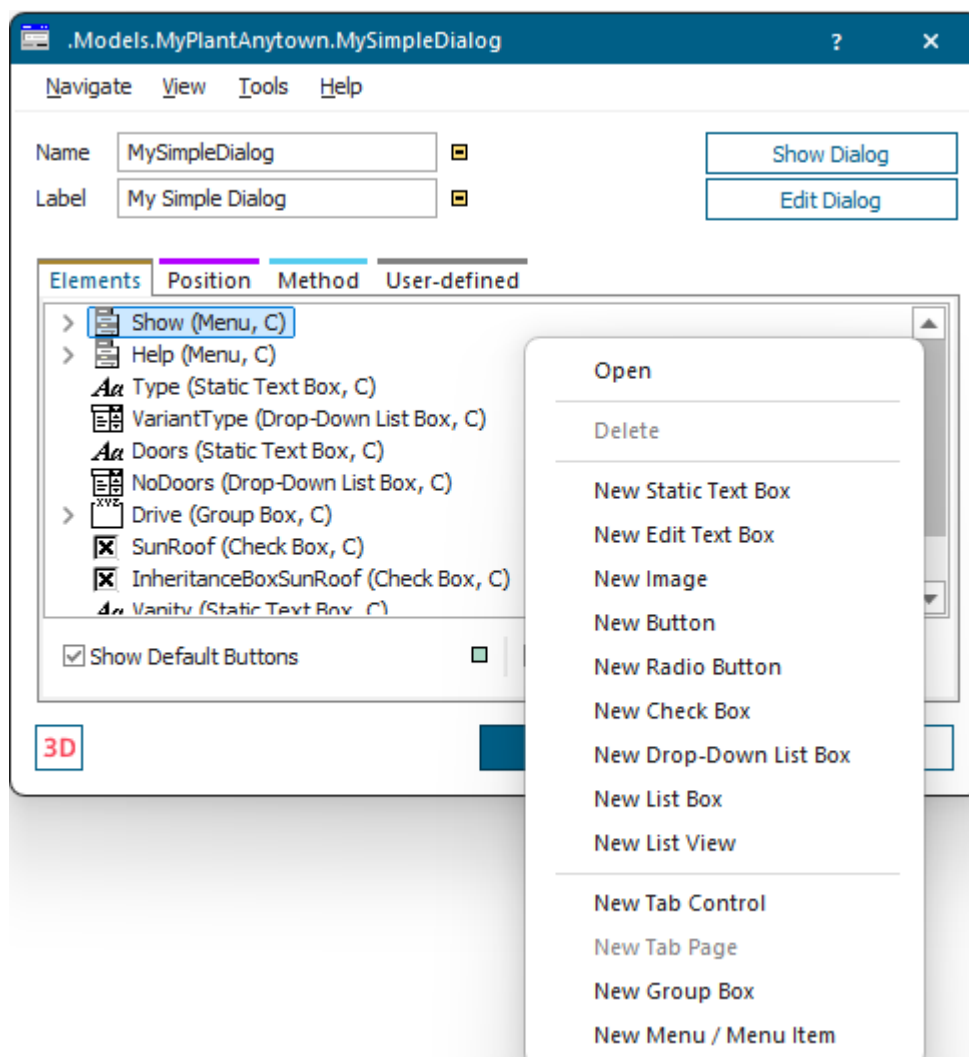
If you want the user to select or to enter several different types of information, you will [Design a Tabbed Dialog](#).

To start designing your own dialog:

- Insert the [Dialog](#)  into the *Frame* in which you are creating your simulation model. You can do this from the folder **UserInterface** in the *Class Library* or from the toolbar **User Interface** in the *Toolbox*.
- Enter a **Name** and a **Label** into the respective text boxes. As opposed to the name, you can enter any term of your choice as the [Label](#). This includes special characters and blank spaces.
 - If you enter a label, *Plant Simulation* show that label in the title bar of your dialog.
 - If you do not enter a label, *Plant Simulation* show the **Name** of the *Dialog* object in the title bar.



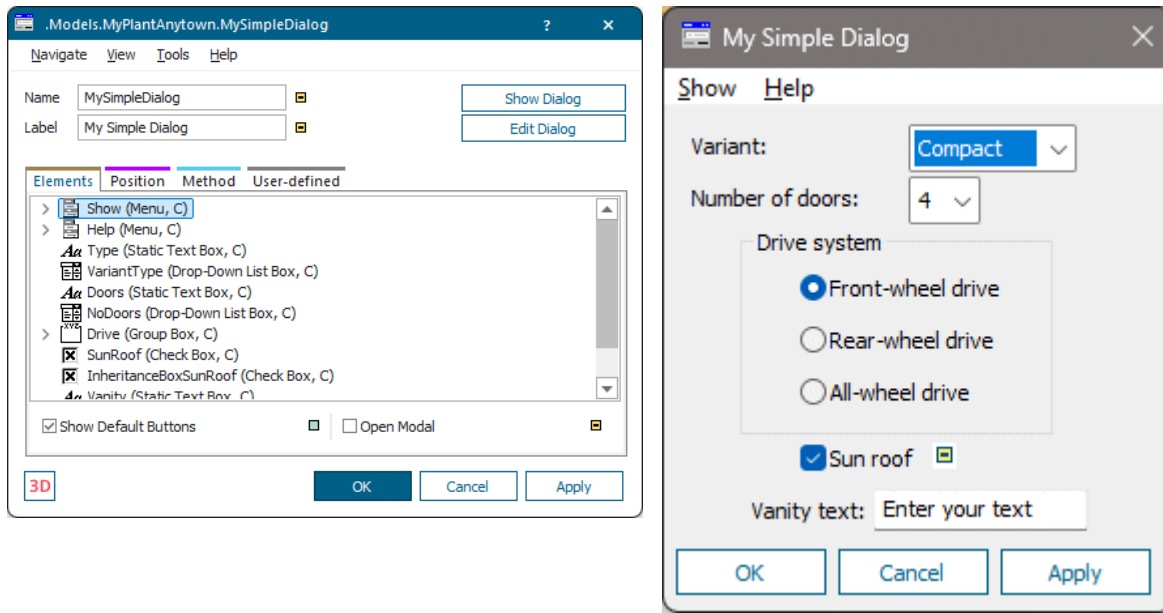
- To insert a dialog item, right-click the tab **Elements** and select the respective dialog item on the context menu.



You can:

Add a Menu and Menu Commands	Add a Set of Radio Buttons
Add a Static Text Box	Add a Check Box
Add a Text Box	Add a List Box
Add a Drop-down List	Add a List View
Add a Group Box Around Dialog Items	Add an Image
Add a Button	

- To show the dialog you are creating with the dialog items you already created, click **Show Dialog**.



- To modify an existing dialog item, right-click it and select **Open** on the context menu. This opens the dialog of the **Dialog Item** where you can change it.
- To delete a dialog item, right-click it and select **Delete** on the context menu.
- To change the Position of a dialog item in the dialog you are creating, first click **Show Dialog** and then click **Edit Dialog**. Select the dialog item you want to move  and drag the mouse to a new location. This way you can, for example, roughly re-position an existing dialog item and then enter the exact coordinates to place it exactly where you want it to be.

Note

The *Dialog* applies the changed coordinates immediately. Once you have moved an item, you cannot undo the move action. To restore the previous settings, you have to re-enter them into the respective dialog.

- To show the standard buttons  in the dialog you are creating, select the check box. To hide these buttons, clear the check box.

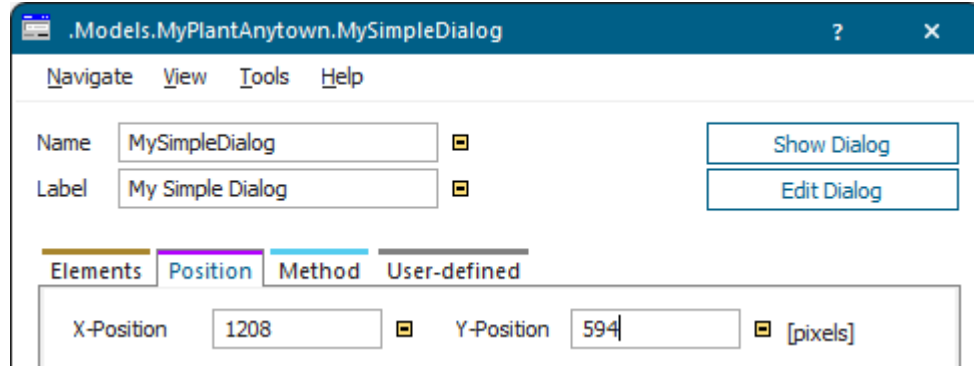
If you do not show this button combination, you have to define your own buttons to provide the user with a way to apply or to discard settings he chose!

- To open the dialog box you are creating modal, select **Open Modal**. Then the user cannot open any other *Plant Simulation* windows until he has completed interacting with the user-defined dialog and closed it.

To allow the user to open other dialog boxes, in addition to the user-defined dialog you are creating, clear the check box.

- To set the position at which the *Dialog* opens on your computer's screen, click the tab **Position** and enter the **X-Position** and the **Y-Position** in pixels into the text boxes. The default setting -1 for both

X-Position and for **Y-Position** centers the *Dialog* on screen. The zero point is located in the top left corner of the screen.



Compare [Program Actions which the Dialog Items Execute](#)

Compare [Program Actions which the Dialog Items Execute](#)

Compare [Program Actions for Interacting with the Dialog](#)

Compare [Create a Dialog Dynamically](#)

Back to [Set Parameters for Objects in Your Own Dialog](#)

Add a Menu and Menu Commands

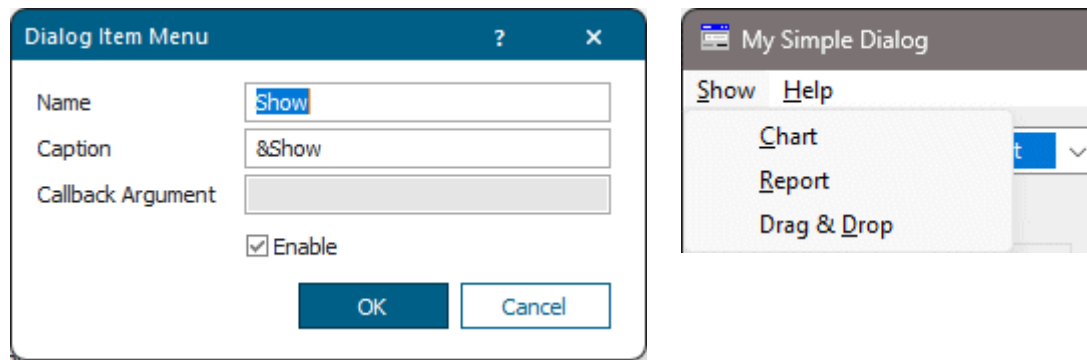
You can add a menu and menu commands to provide access to functions of your dialog.

To add a **menu** to your dialog:

- Right-click the tab **Elements** and select [New Menu/New Menu Command](#).
- Enter the **Name** of the menu. A *Method* object can call this dialog item using the **Name** you entered.
- Enter the **Caption**, which the menu displays. A caption is especially handy if you develop libraries in several languages and want the user to be able to switch languages. In addition you can enter special characters and blanks. In our example we entered &Show. The ampersand & designates the **access key**.

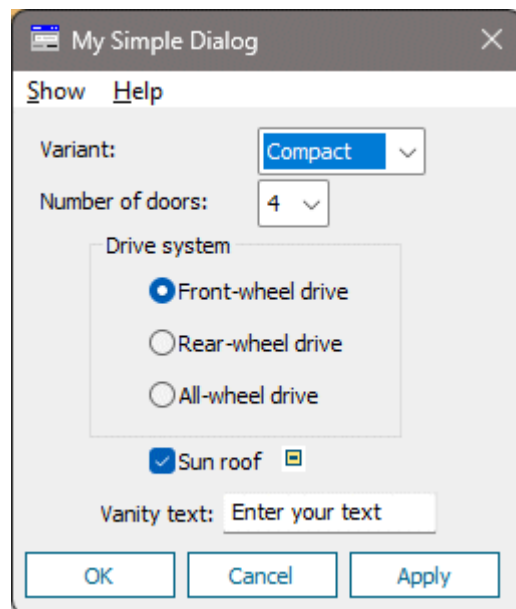
Note

For the top menu itself, you do not have to enter a **Callback Argument**.

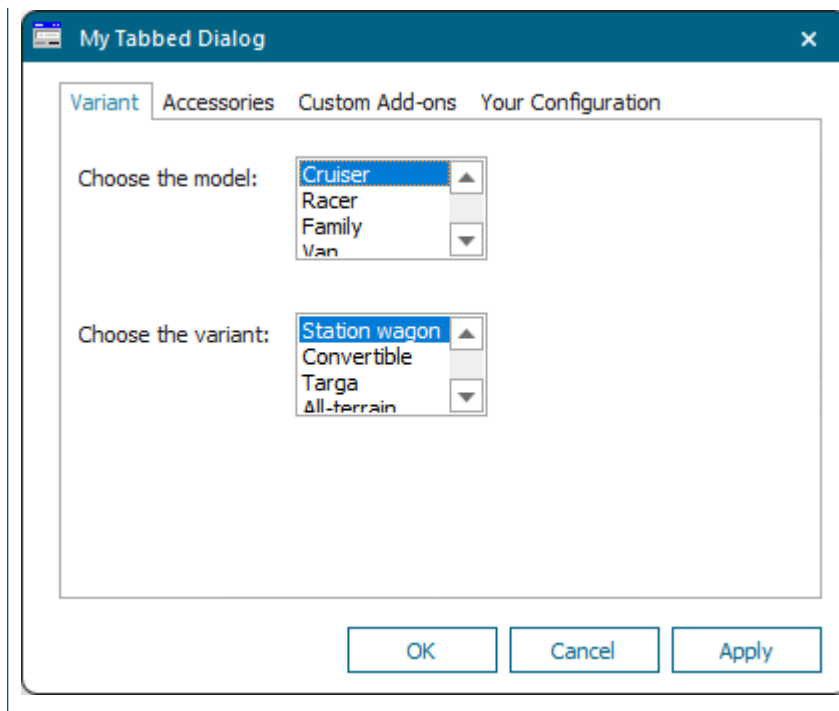


Note

The **display window** of the object *Dialog* that contains menus does not use the standard Siemens PLM theme, but your Windows theme.

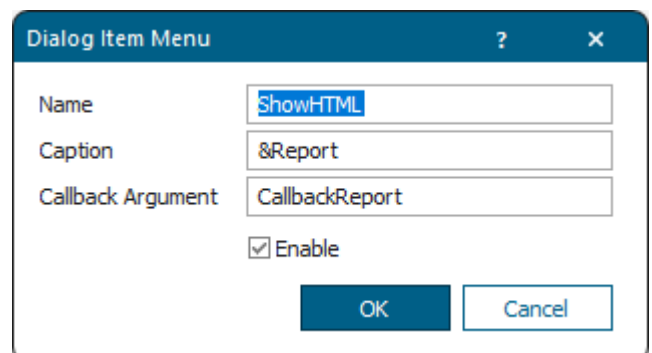
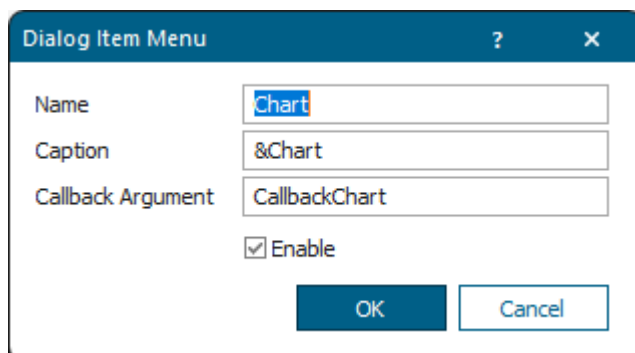


The **display window** of the object *Dialog* that does not contain menus uses the standard Siemens PLM theme.

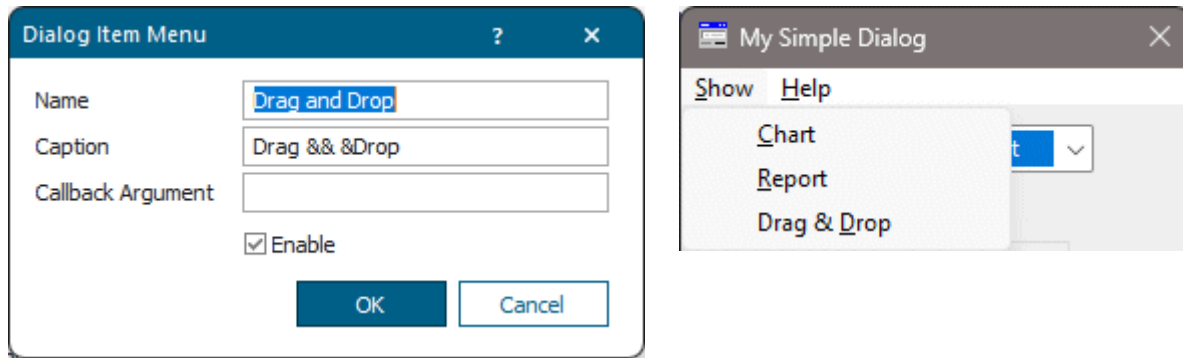


To add a menu command to the menu you added above:

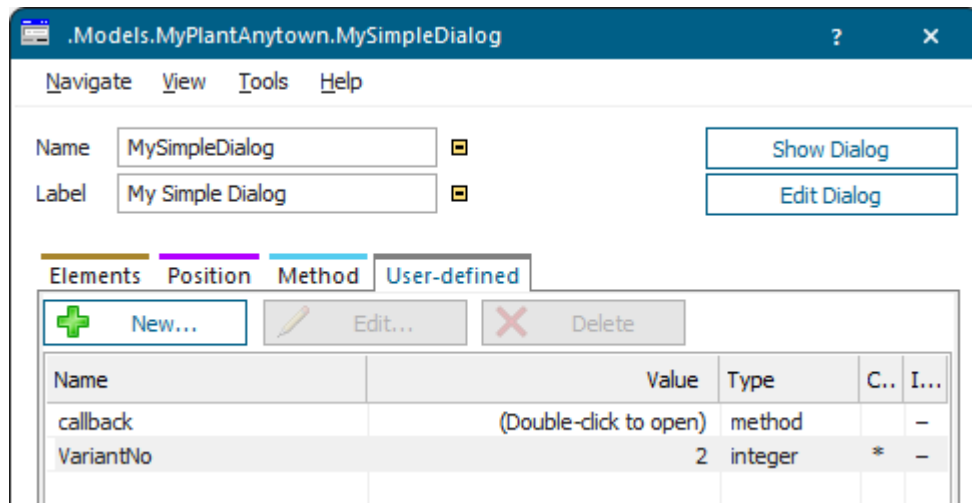
- Right-click the name of the menu on the tab **Elements** and select **New Menu/Menu Command**.
- Enter the **Name** of the menu command.
- Enter the **Caption**, which the menu command displays. In our examples we entered &Chart and &Report. The ampersand & designates the **access key**.



If you want to use the ampersand character & instead of **and** in a dialog item, for example **Drag & Drop**, type in the & twice, Drag && Drop.



- To make that the dialog open the *Chart* or the *Report*, when the user selects **Show > Chart** or **Show > Report**, you have to program it to do so. For this you have to enter the **Callback Argument**, which will be passed to the *callback method*. In our examples we entered `CallbckChart` and `CallbckReport`.
- Then, enter the code for actually opening the *Chart* or the *Report* into the *callback method*. As the *callback method* is a user-defined attribute of data type *method*, click the tab **User-defined**.



Double-click **callback** and click **Open** in the dialog **User-defined Attributes**. To be able to enter commands into the *Method*, click **Inherit Source Code** on the **Edit** ribbon tab of the *Method* so that it is

not selected .

The source code looks like this:

```
param action: string
switch action
case "Open"
    // TODO: add code for the "Open" action here
    // for example ?.setCaption("TextBox", "Test")
    // for example ?.setCheckBox("CheckBox", true)
case "Apply"
    // TODO: add code for the "Apply" action here
    // for example print ?.getValue("TextBox")
```

```

case "Close"      // for example print ?.GetCheckBox("CheckBox")
                  // TODO: add code for the "Close" action here
end

```

Double-click **callback** and click **Open** in the dialog **User-defined Attributes**.

To open the object *MyChart*, we entered:

```

case "CallbackChart"
  MyChart.IsShown := true

```

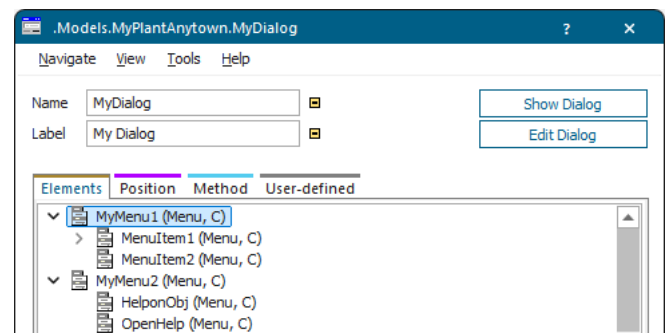
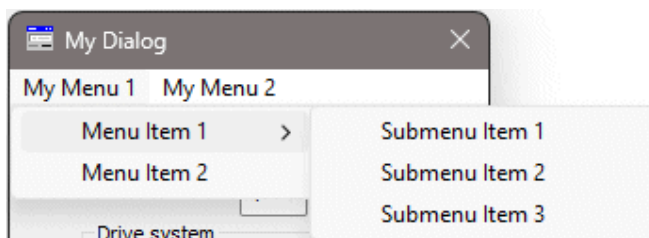
To open the object *MyReport*, we entered:

```

case "CallbackReport"
  MyReport.show

```

- To **change the order** of the **menu commands** on a menu, select a menu command on the tab **Elements**, hold down **Shift** and press the **Up arrow** to move this menu command up or press the **Down arrow** to move this menu command down.
- To add a **submenu** to the selected menu command, right click that menu command and select **New Menu/New Menu Command** again. Repeat the steps described above. A submenu looks like this.



Compare **New Menu/New Menu Command** in the Object Reference

Back to [Design a Simple Dialog](#)

Add a Static Text Box

A **static text box** shows text that the user can view, but not edit.

To add it to your dialog:

- Right-click the tab **Elements** and select **New Static Text Box**.

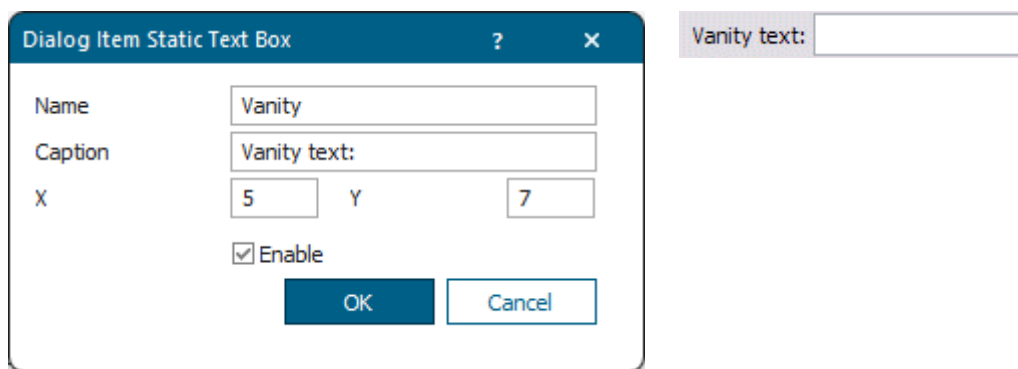
- Enter the **Name** of the static text box. A *Method* object can call this dialog item using the **Name** you entered.
- Enter the **Caption**, which the static text box displays. A caption is especially handy if you develop libraries in several languages and want the user to be able to switch languages. In addition you can enter special characters and blanks.
- Enter the **X**-coordinate and the **Y**-coordinate of the static text box in the *Dialog*.

The position corresponds to the average width of characters of your system font, not to pixels. Changing the font size under **Personalization > Font** affects this setting.

Note

If you cannot see a dialog item after you inserted it, check the settings for the position first. If you entered identical values for different dialog items, the *Dialog* places them on top of each other, so that you can only see the first item(s) you inserted.

- Select if you want to **Enable** the static text box **Vanity text:** or not **Vanity text:**.
- To insert the dialog item into your dialog, click **OK**.



Go to [Program Actions which the Dialog Items Execute](#)

Compare [New Static Text Box](#) in the Object Reference

Back to [Design a Simple Dialog](#)

Add a Text Box

A **text box** is a field into which the user can enter text or edit text it already contains.

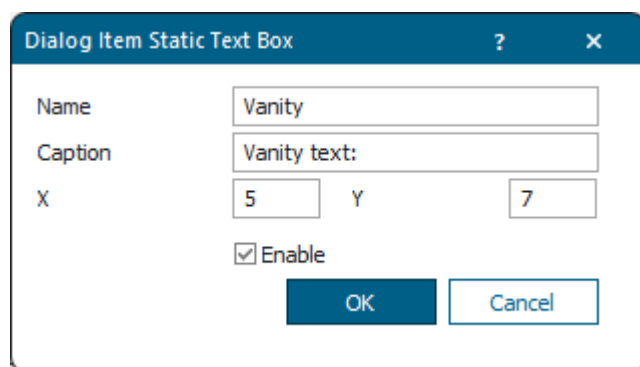
To add it to your dialog:

- Right-click the tab **Elements** and select **New Edit Text Box**.
- Enter the **Name** of the text box. A *Method* object can call this dialog item using the **Name** you entered.
- Enter the **Callback Argument**, which will be passed to the *callback method*.

- Select what kind of data the user can enter from the drop-down list **Data Type**.

Selection	The user can enter
Any Character	any character, which includes special characters, lower and upper case letters and numbers.
Alphanumeric Characters	a blank space, lower and upper case letters, and numbers.
Letters	lower and upper case letters.
Decimal Numbers	0123456789
Signed Decimal Numbers	decimal numbers with a negative or a positive sign: -0123456789 or +0123456789, for example.
Hexadecimal Numbers	ADbf09, for example.
Octal Numbers	01234567, but not 18 for example.
Binary Numbers	0 or 1
Floating Point Numbers	12.3E-43, for example.

- Enter the **X**-coordinate and the **Y**-coordinate of the text box in the *Dialog*.
The position corresponds to the average width of characters of your system font, not to pixels. Changing the font size under **Personalization > Font** affects this setting.
- Enter the **Width** of the text box in the average width of characters of your system font. The default value of 0 sets the width to the values defined by us.
- Select if you want to **Enable** the text box or not. The user can only enter text, when the text box is activated.
- To show text that the user enters into the text box with superscripted lower case x-es , instead of as clear text , select **Password**.
- To insert the dialog item into your dialog, click **OK**.



Vanity text: 

Go to **Program Actions which the Dialog Items Execute**

Compare **New Edit Text Box** in the Object Reference

Back to **Design a Simple Dialog**

Add a Drop-down List

The user can select a single item from the **drop-down list**. When it is closed, it displays the current value.

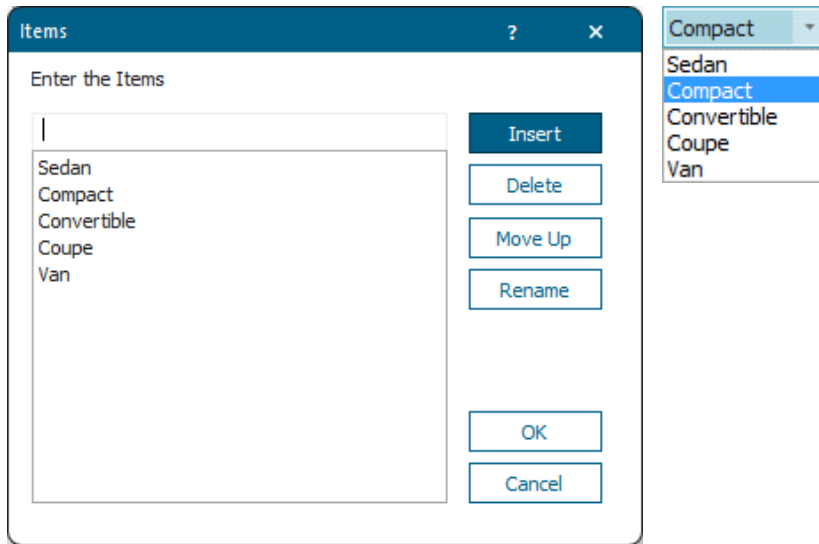
When the user clicks the down arrow  , it opens and displays more items for changing the value.

To add it to your dialog:

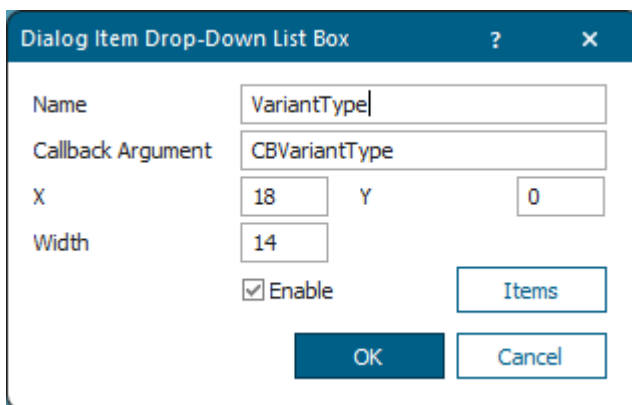
- Right-click the tab **Elements** and select **New DropDownList Box**.
- Enter the **Name** of the drop-down list. A *Method* object can call this dialog item using the **Name** you entered.
- Enter the **Callback Argument**, which will be passed to the *callback method*.
- Enter the **X**-coordinate and the **Y**-coordinate of the drop-down list box in the *Dialog*.

The position corresponds to the average width of characters of your system font, not to pixels. Changing the font size under **Personalization > Font** affects this setting.

- Enter the **Width** of the drop-down list in the average width of characters of your system font. The default value of 0 sets the width to the values defined by us.
- Click **Items** and enter the items, which the list box shows, into the dialog **Items**.
 - To add an item to the list field, enter its name into the text box and click **Insert** or press the **Enter** key.
 - Repeat this for all the items you want to add.
 - To delete an item from the list, select it in the list field and click **Delete**.
 - To move an item up one position in the list, select it in the list field and click **Move Up**.
 - To change the name of an item in the list, select it in the list field, type another name into the text box, and click **Rename**.
 - To apply the items you entered and to close the dialog, click **OK**.
 - To discard what you have done so far, click **Cancel**.



- Select if you want to **Enable** the drop-down list box or not.
- To insert the dialog item into your dialog, click **OK**.



Go to **Program Actions which the Dialog Items Execute**

Compare **New DropDownList Box** in the Object Reference

Back to **Design a Simple Dialog**

Add a Group Box Around Dialog Items

A **group box** graphically groups a set of dialog items or controls by drawing a frame around it. When you want to **group** dialog items, first create the group and then insert and position the individual items within that group.

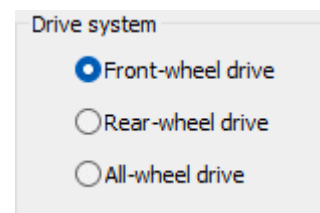
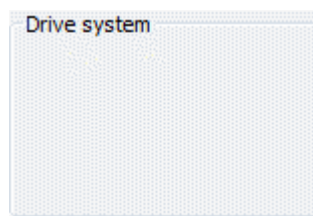
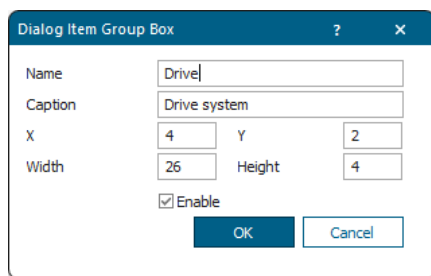
The y-coordinate you specify for items within a group relate to the y-coordinate within that group, not within the overall dialog.

To add a group box to your dialog:

- Right-click the tab **Elements** and select **New Group Box**.
- Enter the **Name** of the group box. A *Method* object can call this dialog item using the **Name** you entered.
- Enter the **Caption**, which the group box displays. A caption is especially handy if you develop libraries in several languages and want the user to be able to switch languages. In addition you can enter special characters and blanks.
- Enter the **X**-coordinate and the **Y**-coordinate of the group box in the *Dialog*.

The position corresponds to the average width of characters of your system font, not to pixels. Changing the font size under **Personalization > Font** affects this setting.

- Enter the **Width** and the **Height** of the group box in the average width of characters of your system font. The default value of 0 sets the width to the values defined by us.
- Select if you want to **Enable** the group box or not.
- To insert the dialog item into your dialog, click **OK**.



- To change the order of the items within a **group**, select an item within the group on the tab **Elements**, hold down **Shift** and press the **Up arrow** to move this item up or press the **Down arrow** to move this item down.

Go to [Program Actions which the Dialog Items Execute](#)

Compare [New Group Box](#) in the Object Reference

Back to [Design a Simple Dialog](#)

Add a Set of Radio Buttons

A set of **radio buttons** enables the user to select a single setting from a fixed set of choices that exclude each other.

To add it to your dialog:

- Right-click the tab **Elements** and select **New Radio Button**.

- Enter the **Name** of the radio button. A *Method* object can call this dialog item using the **Name** you entered.
- Enter the **Caption** that describes what the radio button does and that the *Dialog* shows. A caption is especially handy if you develop libraries in several languages and want the user to be able to switch languages. In addition, you can enter special characters and blanks.
- Enter the **Callback Argument**, which will be passed to the *callback method*.
- Enter the **X**-coordinate and the **Y**-coordinate of the radio button in the *Dialog*.

The position corresponds to the average width of characters of your system font, not to pixels. Changing the font size under **Personalization > Font** affects this setting.

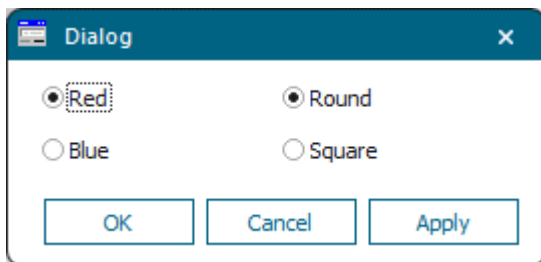
Note

If you cannot see a dialog item after you inserted it, check the settings for the position first. If you entered identical values for different dialog items, the *Dialog* places them on top of each other, so that you can only see the first item(s) you inserted.

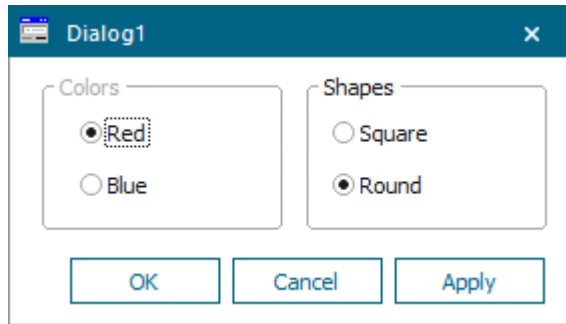
- Enter a group id, i.e., a number, for grouping any number of radio buttons to denote which radio buttons belong together. **Group ID 1** might stand for one set of radio buttons, while **Group ID 2** might stand for another set. The user can only select one radio button of a group at a time.

Note

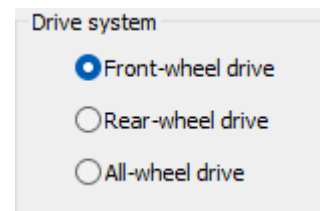
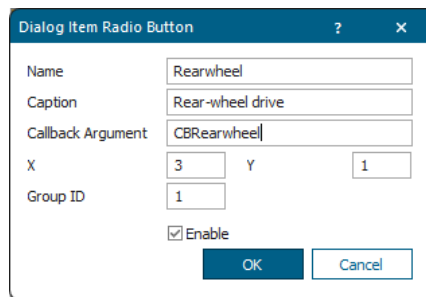
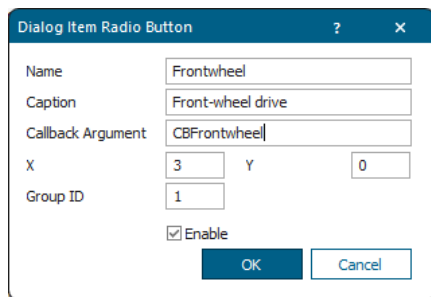
You have to create all items of a group of radio buttons sequentially because *Plant Simulation* creates the items according to their row and column position. You cannot arrange two sets of radio buttons as shown below, where the radio buttons **Red** and **Blue** on the left-hand side and the radio buttons **Round** and **Square** on the right-hand side form a group each. Windows ignores the **Group ID** you enter and groups the radio buttons row by row according to the **Y**-coordinate, i.e., it treats **Red** and **Round** as one group and **Blue** and **Square** as the other group.



If you want to model like this, do not place the radio buttons loosely onto the dialog, but group them within group boxes.



- Select if you want to **Enable** the radio button or not.
- To insert the dialog item into your dialog, click **OK**.
- Repeat these steps to add additional radio buttons.



Go to **Program Actions which the Dialog Items Execute**

Compare **New Radio Button** in the Object Reference

Back to **Design a Simple Dialog**

Add a Check Box

A **check box** displays a setting or a set of settings that do not exclude each other. The user can select or clear one or several check boxes at the same time.

To add it to your dialog:

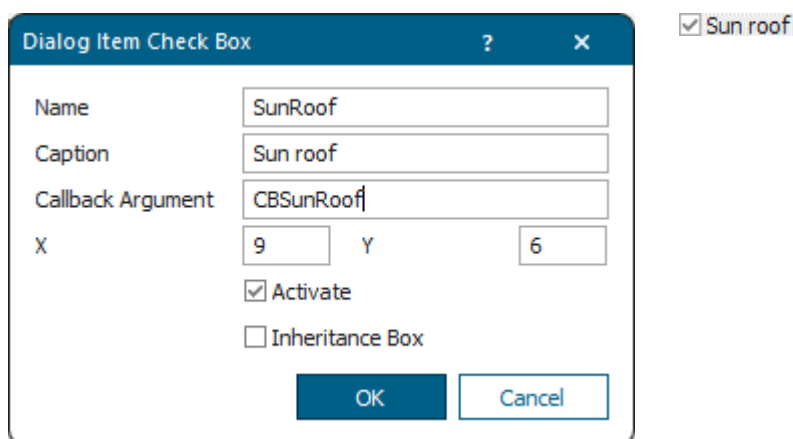
- Right-click the tab **Elements** and select **New Check Box**.
- Enter the **Name** of the check box. A *Method* object can call this dialog item using the **Name** you entered.
- Enter the **Caption** that describes what the check box does. A caption is especially handy if you develop libraries in several languages and want the user to be able to switch languages. In addition you can enter special characters and blanks.
- Enter the **Callback Argument**, which will be passed to the *callback method*.
- Enter the **X**-coordinate and the **Y**-coordinate of the check box in the *Dialog*.

The position corresponds to the average width of characters of your system font, not to pixels. Changing the font size under **Personalization > Font** affects this setting.

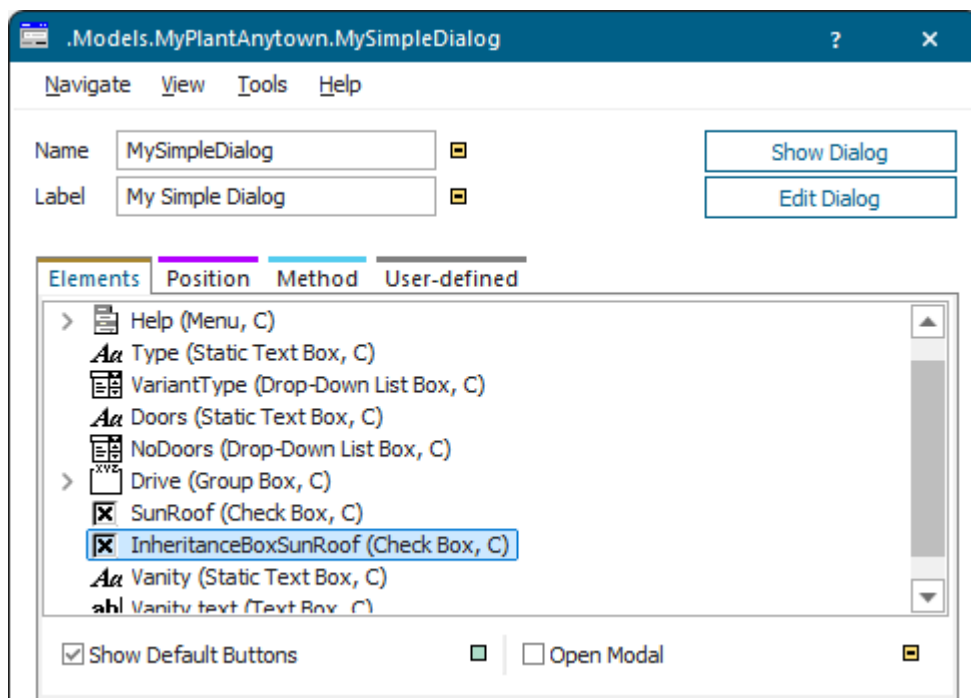
Note

If you cannot see a dialog item after you inserted it, check the settings for the position first. If you entered identical values for different dialog items, the *Dialog* places them on top of each other, so that you can only see the first item(s) you inserted.

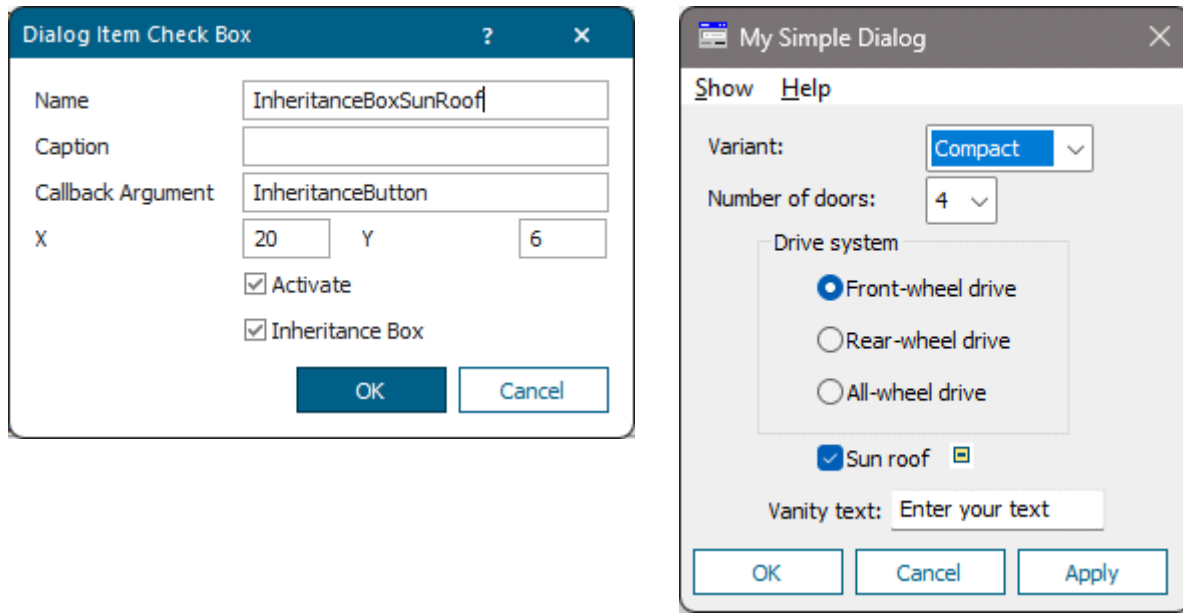
- Select if you want to **Enable** the check box or not.
- To insert the dialog item into your dialog, click **OK**.



- To insert an **Inheritance box** to the right of our check box named **Sun roof**, we create a new check box.



We selected the settings below.



Go to [Program Actions which the Dialog Items Execute](#)

Compare [New Check Box](#) in the Object Reference

Back to [Design a Simple Dialog](#)

Design a Tabbed Dialog

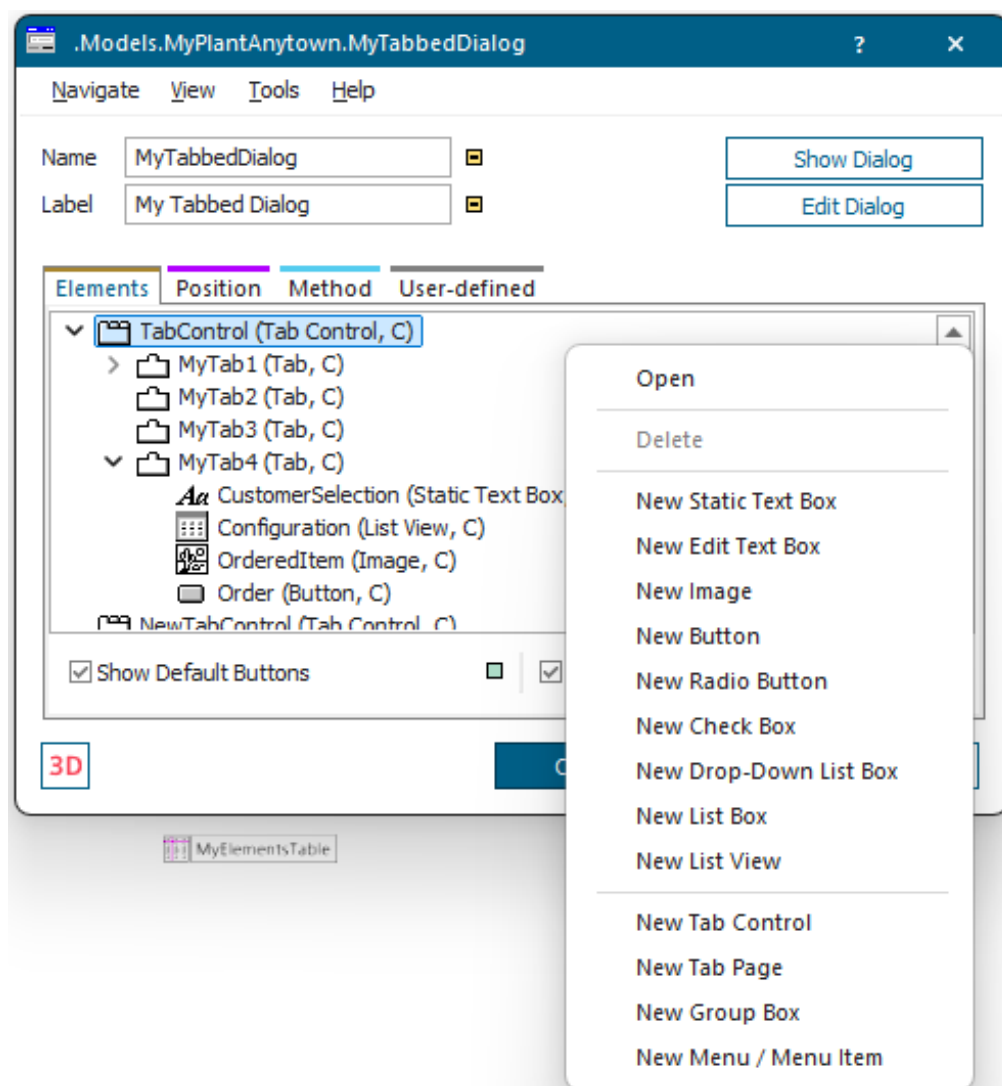
Design a Tabbed Dialog

If you want the user to select or to enter several different types of information, you will create a tabbed dialog.

If you only need the user to select or to enter a small number of items, you can present them directly in the dialog in a flat structure, compare [Design a Simple Dialog](#).

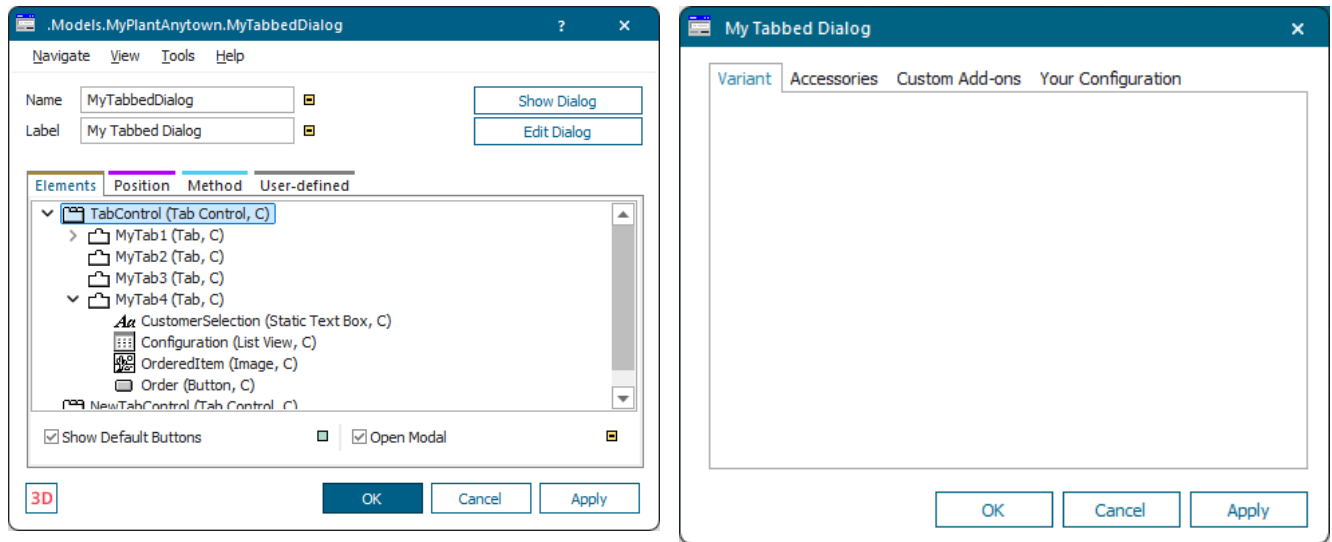
To start designing your own dialog:

- Insert the [Dialog](#)  into the *Frame* in which you are creating your simulation model. You can do this from the folder **UserInterface** in the *Class Library* or from the toolbar **User Interface** in the *Toolbox*.
- Enter a **Name** and a **Label** into the respective text boxes. As opposed to the name, you can enter any term of your choice as the label. This includes special characters and blank spaces.
 - If you enter a **Label**, *Plant Simulation* shows that label in the title bar of your dialog.
 - If you do not enter text for the label, *Plant Simulation* shows the **Name** of the *Dialog* object in the title bar.
- To insert a dialog item, right-click the tab **Elements** and select a dialog item on the context menu.



You can:

Add a Menu and Menu Commands	Add a Set of Radio Buttons
Add a Static Text Box	Add a Check Box
Add a Text Box	Detach the Camera from an Object
Add a Drop-down List	Add a List Box
Add a Group Box Around Dialog Items	Add an Image
Add a Button	Add a Tab Control
Add Tabs to a Tab Control	



- To modify an existing dialog item, right-click it and select **Open** on the context menu. This opens the dialog of the **Dialog Item** where you can change it.
- To delete a dialog item, right-click it and select **Delete** on the context menu.
- To show the dialog you are creating, click **Show Dialog**.
- To change the position of a dialog item in the dialog you are creating, first click **Show Dialog** and

then click **Edit Dialog**. Select the dialog item you want to move  and drag the mouse to a new location. This way you can, for example, roughly re-position an existing dialog item and then enter the exact coordinates to place it exactly where you want it to be.

Note

The *Dialog* applies the changed coordinates immediately. When you have moved an item, you cannot undo the move action. To restore the previous settings, you have to re-enter them into the respective dialog.

- To show the standard buttons    in the dialog you are creating, select the check box. To hide these buttons, clear the check box.

If you do not show this button combination, you have to define your own buttons to provide the user with a way to apply or to discard settings he chose.

- To open the dialog box you are creating modal, select **Open Modal**. Then the user cannot open any other *Plant Simulation* windows until he has completed interacting with the user-defined dialog and closed it.

To allow the user to open other dialog boxes, in addition to the user-defined dialog you are creating, clear the check box.

- To set the position at which the *Dialog* opens on your computer's screen, click the tab **Position** and enter the **X-Position** and the **Y-Position** in pixels into the text boxes. The default setting -1 for both

X-Position and for **Y-Position** centers the *Dialog* on screen. The zero point is located in the top left corner of the screen.

Compare [Program Actions which the Dialog Items Execute](#)

Compare [Program Actions which the Dialog Items Execute](#)

Compare [Program Actions for Interacting with the Dialog](#)

Compare [Create a Dialog Dynamically](#)

Back to [Set Parameters for Objects in Your Own Dialog](#)

Add a Tab Control

If you are going to create a tabbed dialog, you first have to create the **Tab Control**. The tab control resembles a notebook that contains any number of pages (tabs). The user can navigate between different tabs and/or sections of information within the dialog box.

Note

Do not confuse the tab control with the tabs themselves. The tab control is the container that holds the individual tabs (pages).

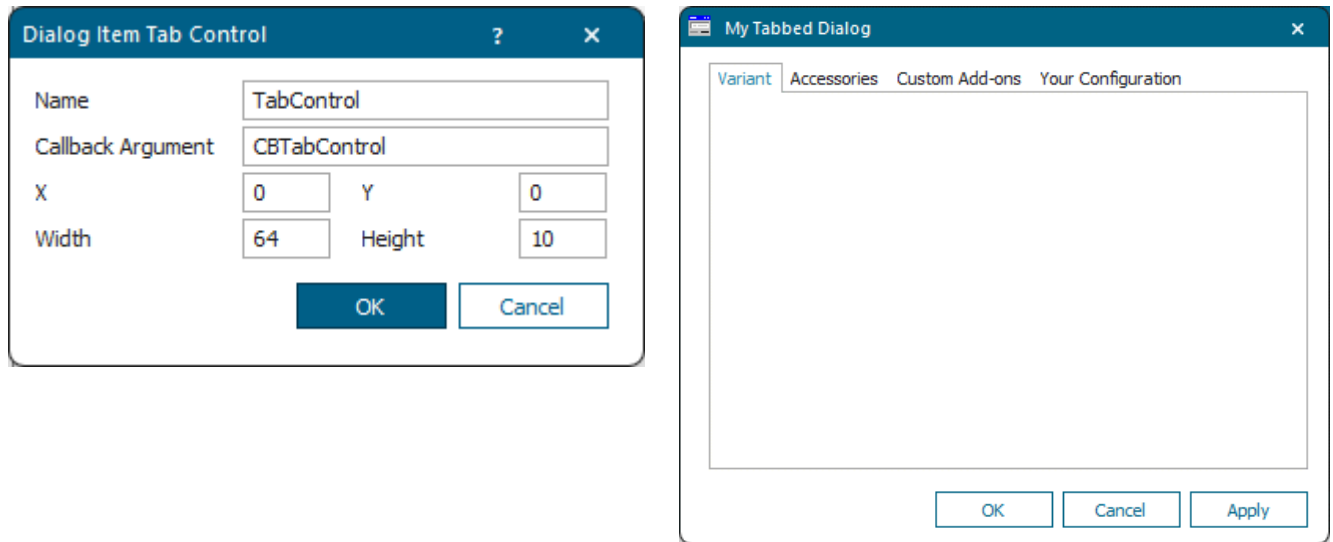
When the user selects another tab, the *Dialog* calls the [Callback Method](#), which in turn executes the actions you programmed with the first parameter you typed in as [Callback Argument](#).

To add it to your dialog:

- Right-click the tab **Elements** and select [New Tab Control](#).
- Type in the [Name](#) of the tab control. A *Method* object can call this dialog item using the **Name** you typed in.
- Type in the [Callback Argument](#), which will be passed to the *callback method*.
- Type in the [X](#)-coordinate and the [Y](#)-coordinate of the tab control in the *Dialog*.

The position corresponds to the average width of characters of your system font, not to pixels. Changing the font size under **Personalization > Font** affects this setting.

- Type in the **Width** and the **Height** of the tab control in the *Dialog*. The default value of 0 sets the height and the width to the values defined by us.
- To insert the **Tab Control** into your dialog, click **OK**.



- To add one **Tab** at the time to the **Tab Control**, click it with the right mouse button on the tab **Elements** and select **New Tab Page** on the context menu.
- To change the order of the tabs in the dialog, select a tab  on the tab **Elements**, hold down **Shift** and press the **Up arrow** to move this tab to the left or press the **Down arrow** to move this tab to the right. The dialog shows the first tab in the structure as the leftmost tab.

Go to [Program Actions which the Dialog Items Execute](#)

Compare [New Tab Control](#) in the Object Reference

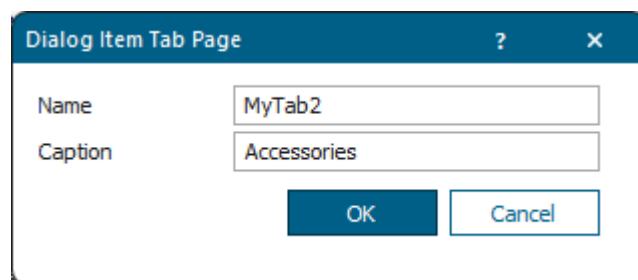
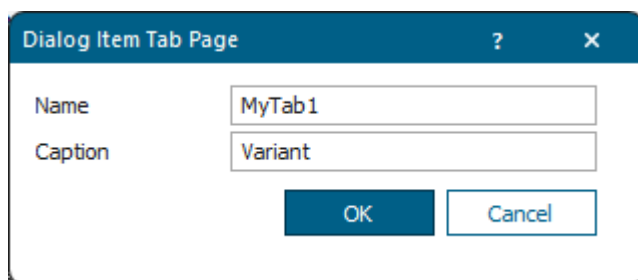
Back to [Design a Tabbed Dialog](#)

Add Tabs to a Tab Control

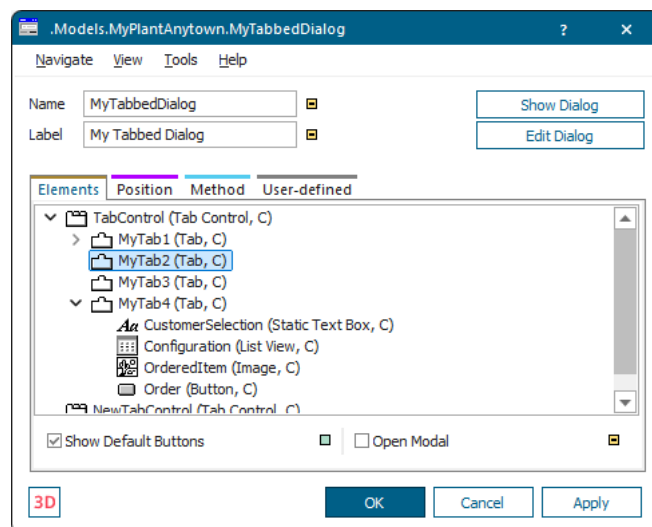
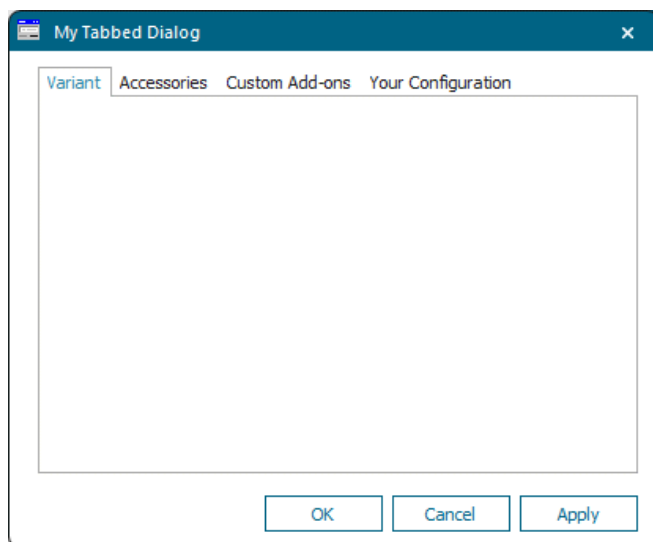
You can add individual tabs to the **Tab Control**, which is the container for the tabs.

To add one **Tab** at a time to the **Tab Control**:

- Right-click the tab **Elements** and select **New Tab Page**.
- Enter the **Name** of the tab into the dialog **Dialog Item Tab Page**. A *Method* object can call this dialog item using the **Name** you entered.
- Enter the **Caption** of the tab. A caption is especially handy if you develop libraries in several languages and want the user to be able to switch languages. In addition you can enter special characters and blanks. The tab on the tab control shows this caption as the title on the tab itself.
- To insert the dialog item into your dialog, click **OK**.



- To change the order of the tabs in the dialog, select a tab on the tab **Elements**, hold down **Shift** and press the **Up arrow** to move this tab to the left or the **Down arrow** to move it to the right. The dialog shows the first tab in the structure as the leftmost tab.



Compare [Add a Tab Control](#)

Go to [Program Actions which the Dialog Items Execute](#)

Compare [New Tab Page](#) in the Object Reference

Back to [Design a Tabbed Dialog](#)

Add a List Box

A **List Box** displays a list of choices. The user can double-click one of the items in the list to select it. As opposed to the drop-down list box, the list box has a fixed size and does not collapse.

To add it to your dialog:

- Right-click the tab **Elements** and select [New List Box](#).

- Type in the **Name** of the list box. A *Method* object can call this dialog item using the **Name** you typed in.
- Type in the **Callback Argument**, which will be passed to the *callback method*.
- Type the **X**-coordinate and the **Y**-coordinate of the list box in the *Dialog*.

The position corresponds to the average width of characters of your system font, not to pixels. Changing the font size under **Personalization > Font** affects this setting.

Note

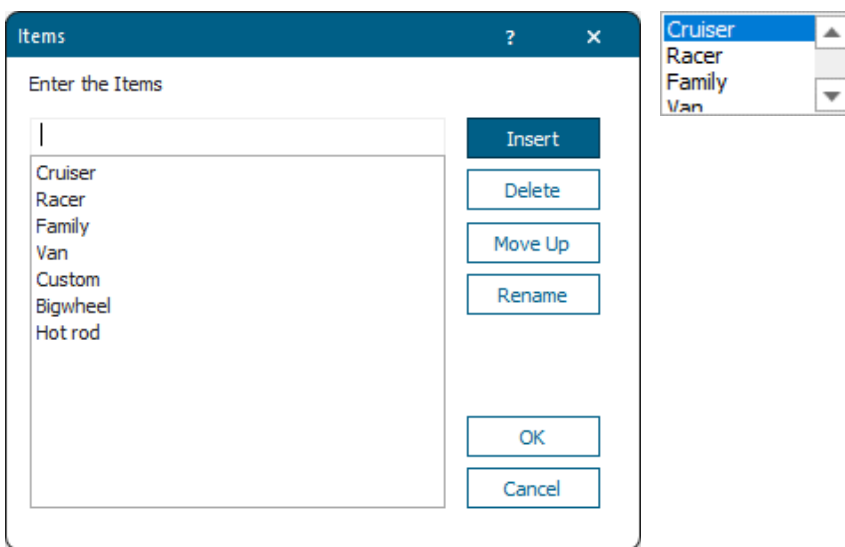
If you cannot see a dialog item after you inserted it, check the settings for the position first. If you typed in identical values for different dialog items, the *Dialog* places them on top of each other, so that you can only see the first item(s) you inserted.

- Type the **Width** and the **Height** of the list box in the *Dialog*. If you do not type in values, the *Dialog* sets the width and height to the values defined by us. When you type in a value for the height, it adds a vertical scrollbar to the list, if the list contains more items than the height can display.

The screenshot shows the 'Dialog Item List Box' dialog box. It has a title bar with a question mark and a close button. The dialog contains the following fields and controls:

- Name:** A text box containing 'ModellList'.
- Callback Argument:** A text box containing 'CBModellList'.
- X:** A text box containing '18'.
- Y:** A text box containing '0'.
- Width:** A text box containing '16'.
- Height:** A text box containing '2'.
- Enable:** A checkbox that is checked.
- Items:** A button to open the list of items.
- OK:** A button to confirm the settings.
- Cancel:** A button to discard the settings.

- Click **Items** and type the items the list box displays in the dialog **Items**.
 - To add an item to the list field, type its name into the text box and click **Insert** or press the **Enter** key.
 - Repeat this for all the items you want to add.
 - To delete an item from the list, select it in the list field and click **Delete**.
 - To move an item up one position in the list, select it in the list field and click **Move Up**.
 - To change the name of an item in the list, select it in the list field, type another name into the text box, and click **Rename**.
 - To apply the items you typed in and to close the dialog, click **OK**.
 - To discard what you have done so far, click **Cancel**.



- Select if you want to **Enable** the list box or not.
- To insert the dialog item into your dialog, click **OK**.

Go to **Program Actions which the Dialog Items Execute**

Compare **New List Box** in the Object Reference

Back to **Design a Tabbed Dialog**

Add a List View

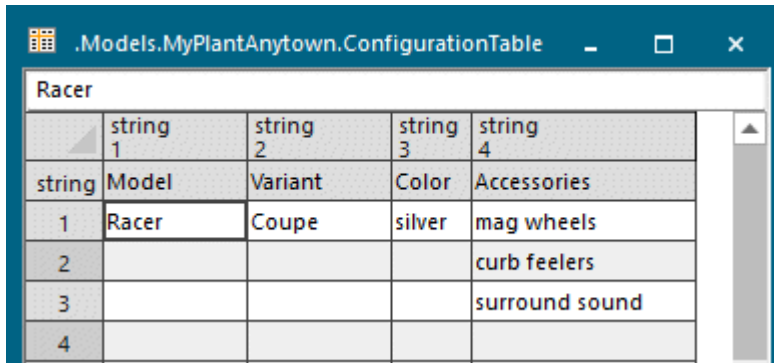
A **List View** displays a table in the dialog.

Note

When displaying *strings*, *Plant Simulation* only shows the first 260 characters.

To add it to your dialog:

- Right-click the tab **Elements** and select **New List View**.
- Enter the **Name** of the list view. A *Method* object can call this dialog item using the **Name** you entered.
- Enter the **Callback Argument**, which will be passed to the **Callback Method**.
- Enter the name of a table or click the button  and select a table in the dialog **Select Object**.
 - Activate the column index in the *DataTable* and enter the column headings which the list view shows into the header cells.
 - Enter the items which the list view shows into the cells of the columns.

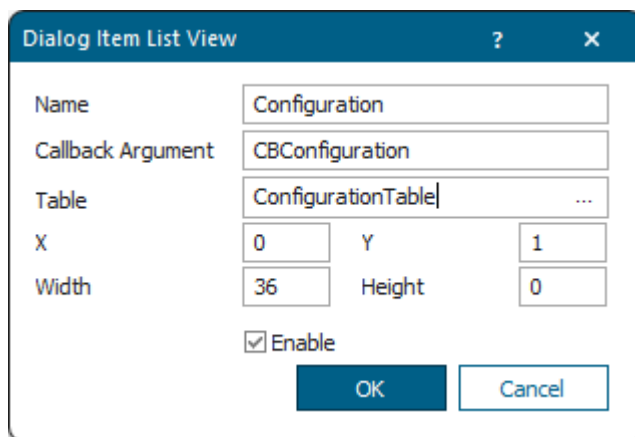


	string 1	string 2	string 3	string 4
string	Model	Variant	Color	Accessories
1	Racer	Coupe	silver	mag wheels
2				curb feelers
3				surround sound
4				

- Enter the **X**-coordinate and the **Y**-coordinate of the list view in the *Dialog*.

The position corresponds to the average width of characters of your system font, not to pixels. Changing the font size under **Personalization > Font** affects this setting.

- Enter the **Width** and the **Height** of the list view in the *Dialog*.
- Select if you want to **Enable** the list view or not.
- To insert the dialog item into your dialog, click **OK**.



Dialog Item List View

Name: Configuration

Callback Argument: CBConfiguration

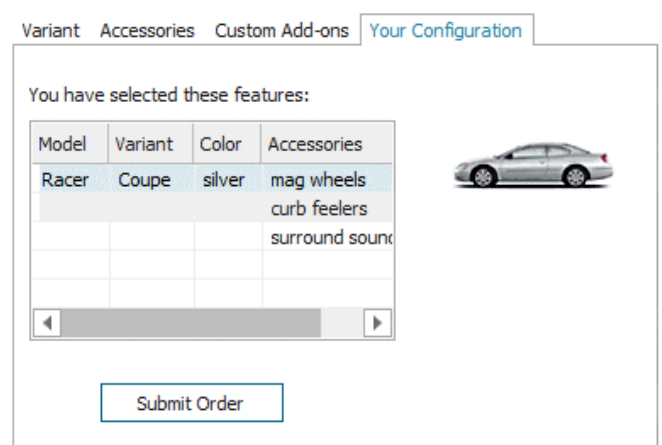
Table: ConfigurationTable

X: 0 Y: 1

Width: 36 Height: 0

☒ Enable

OK Cancel



Variant Accessories Custom Add-ons Your Configuration

You have selected these features:

Model	Variant	Color	Accessories
Racer	Coupe	silver	mag wheels
			curb feelers
			surround sound

Submit Order

Go to **Program Actions which the Dialog Items Execute**

Compare **New List View** in the **Object Reference Help**

Back to **Design a Tabbed Dialog**

Add a Button

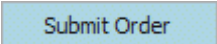
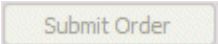
When the user clicks a **button**, it calls the **Callback Method** with the parameter you entered as the **Callback Argument**.

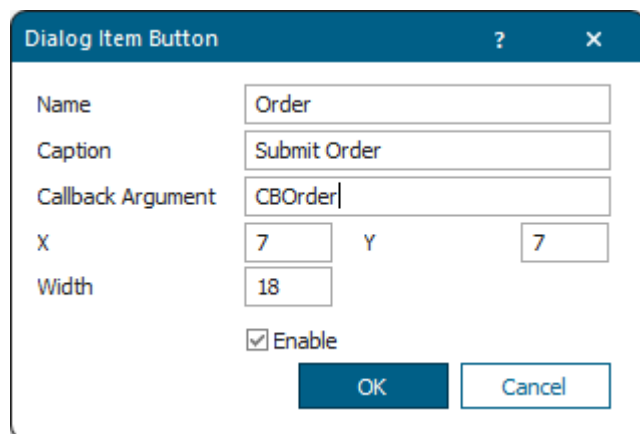
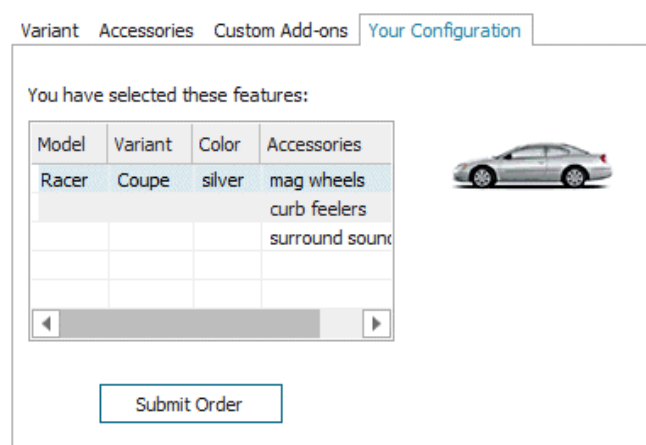
To add it to your dialog:

- Right-click the tab **Elements** and select **New Button**.

- Enter the **Name** of the button. A *Method* object can call this dialog item using the **Name** you entered.
- Enter the **Caption**, which the button displays. A caption is especially handy if you develop libraries in several languages and want the user to be able to switch languages. In addition you can enter special characters and blanks.
- Enter the **Callback Argument** into the text box. It in turn executes the actions you programmed in the **Callback Method**.
- Enter the **X**-coordinate and the **Y**-coordinate of the button in the *Dialog*.

The position and the width correspond to the average width of characters of your system font, not to pixels. Changing the font size under **Personalization > Font** affects this setting.

- Enter the **Width** of the button in the *Dialog*. The default value of 0 automatically sets the width of the button to the width of the **OK** button.
- Select if you want to **Enable** the button  or not .
- To insert the dialog item into your dialog, click **OK**.

Model	Variant	Color	Accessories
Racer	Coupe	silver	mag wheels
			curb feelers
			surround sound

Go to **Program Actions which the Dialog Items Execute**

Compare **New Button** in the Object Reference

Back to **Design a Tabbed Dialog**

Add an Image

An **Image** is a picture, i.e., an icon you defined for the object *Dialog*. You can enter a number, or a name, such as Icon1.

To add it to your dialog:

- Right-click the tab **Elements** and select **New Image**.
- Enter the **Name** of the image. A *Method* object can call this dialog item using the **Name** you entered.

- Enter the **X**-coordinate and the **Y**-coordinate of the image in the *Dialog*.

The position corresponds to the average width of characters of your system font, not to pixels. Changing the font size under **Personalization > Font** affects this setting.

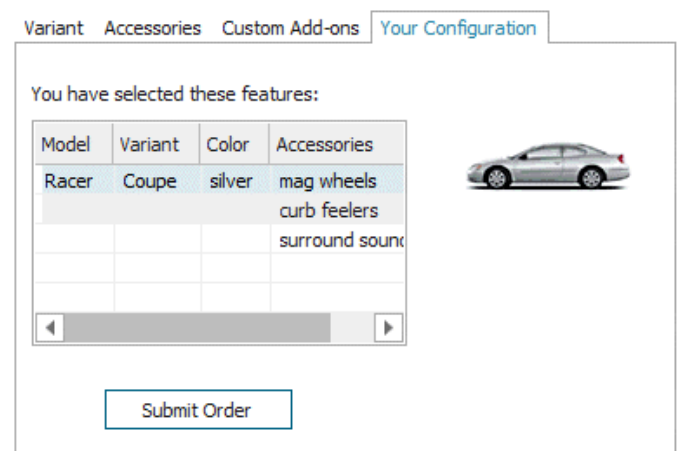
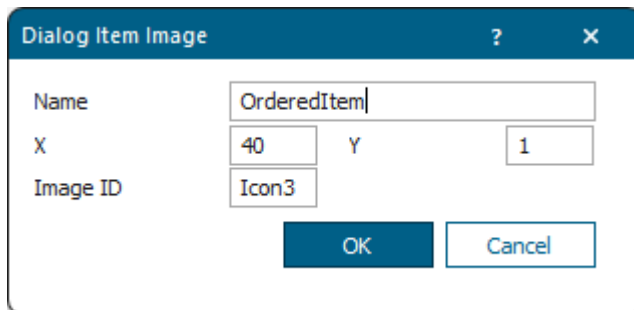
Note

If you cannot see a dialog item after you inserted it, check the settings for the position first. If you entered identical values for different dialog items, the *Dialog* places them on top of each other, so that you can only see the first item(s) you inserted.

- Enter the **Width** and the **Height** of the image in the *Dialog*.
- Enter an **Image ID**, i.e., the number of the icon, or an image name. This image is an icon of the *Dialog*.
- To insert the dialog item into your dialog, click **OK**.

Note

Plant Simulation shows the dialog with the settings you select under **Personalization > Font** as well as with the settings you select under **Display settings > Scale & layout**. When you change these settings, the dialog scales text, while the images always retain their size in pixels. For this reason it can happen that text overlaps the image. We recommend to test your dialogs with different display settings to make sure that everything works as you expect it.



Go to [Program Actions which the Dialog Items Execute](#)

Compare [Create an Icon](#)

Compare [New Image](#) in the **Object Reference Help**

Back to [Design a Tabbed Dialog](#)

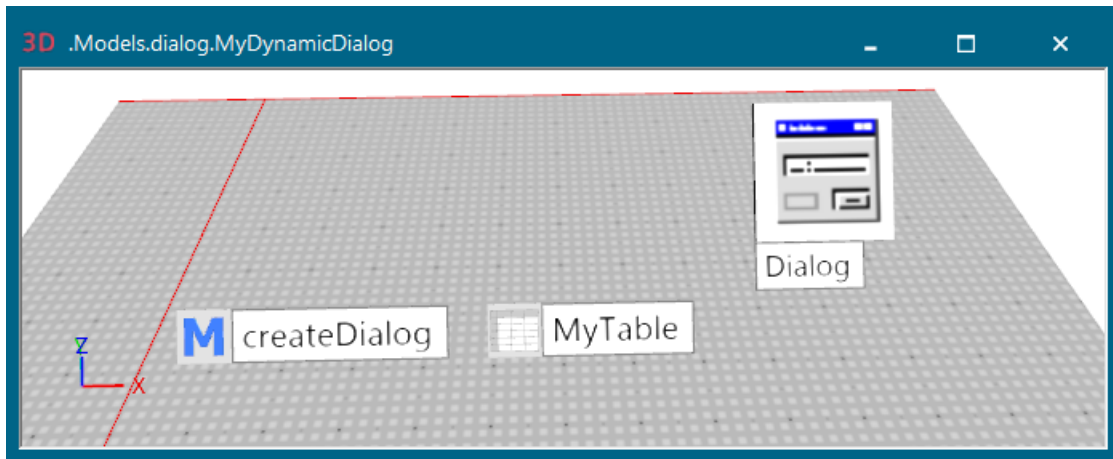
Create a Dialog Dynamically

You can also create a *Dialog* dynamically in your simulation model depending on your modeling needs.

Steps to Create a Dynamic Dialog

We proceeded as follows to create our sample model:

- We inserted a *Method*, a *DataTable*, and a *Dialog* into our model. We named the model MyDynamicDialog and the objects as shown in the screenshot.



- We programmed what the three **tabs** in the *Dialog* are going to show in the *Method* named createDialog.

```
Dialog.Label := "My Dynamic Dialog"; print Dialog.Label
var testBool:boolean := true
Dialog.clearData
testBool := testBool AND Dialog.createTabControl("TabControl", 0, 0, 45,
10)
testBool := testBool AND Dialog.createTabPage("Tab 1","TabControl")
    testBool := testBool AND Dialog.createTabPage("Tab 2","TabControl")
    testBool := testBool AND Dialog.createTabPage("Tab 3","TabControl")
-- Menus
testBool := testBool AND Dialog.createMenu("Show")
testBool := testBool AND Dialog.createMenu("Item 1","Show")
testBool := testBool AND Dialog.createMenu("Item 2","Show")
dialog.setCallbackArgument("Item 1","Item 1 Arg")
dialog.setCallbackArgument("Item 2","Item 2 Arg")
-- Groups on different pages
testBool := testBool AND Dialog.createGroupBox("Group 1", 0,1, 40,3,"Tab
1")
testBool := testBool AND Dialog.createGroupBox("Group 2", 0,4, 40,4,"Tab
1")
testBool := testBool AND Dialog.createGroupBox("Group 1", 0,0, 40,3,"Tab
2")
testBool := testBool AND Dialog.createGroupBox("Group 2", 0,4, 18,4,"Tab
2")
```

```

testBool := testBool AND Dialog.createGroupBox("Group 3", 22,4, 18,4,"Tab
2")
-- static text outside a group
testBool := testBool AND Dialog.createStaticTextBox("Static text",
0,0,"Tab 1")
testBool := testBool AND Dialog.createStaticTextBox("Static text",
0,0,"Tab 3")
Dialog.setCaption("Tab 3.Static text","Double-click a row ...")
-- static text inside a group
testBool := testBool AND Dialog.createStaticTextBox("Click the button",
0,0,"Tab 1.Group 2")
-- Checkboxes
testBool := testBool AND Dialog.createCheckBox(" Check box 1", 15,0,"Tab
1")
testBool := testBool AND Dialog.createCheckBox(" Check box 2",
20,0,"Group 2")
--print Dialog.createCheckBox(" Check box 2", 20,0,"Tab 2.Group 2");
testBool := testBool AND dialog.setCaption(" Check box 2","Checkbox Gr
2")-- true
dialog.setCallbackArgument(" Check box 2","CheckboxGr2")
-- checkbox with action
testBool := testBool AND Dialog.createCheckBox("Check box", 1,0,"Tab
1.Group 1")
testBool := testBool AND Dialog.setCaption("Check box","Active")
dialog.setCallbackArgument("Check box","Check box")
-- Buttons
testBool := testBool AND Dialog.createButton("My_Button", 0, 1, 13,
"Group 2")
testBool := testBool AND dialog.setCaption("My_Button","Security")-- true
dialog.setCallbackArgument("My_Button","Button Arg")

testBool := testBool AND Dialog.createButton("A_Button", 0, 1, 13, "Tab
1.Group 1")
testBool := testBool AND Dialog.setCaption("A_Button","Attention")
Dialog.setSensitive("A_Button",false)

-- Radiobuttons and their groupings in a single group
-- note that radionbuttons of a group must be in a row of the dialog
testBool := testBool AND Dialog.createRadioButton("RadioButton 1", 3,
0,"Tab 2.Group 1")
testBool := testBool AND Dialog.createRadioButton("RadioButton 2", 20,
0,"Tab 2.Group 1")
testBool := testBool AND Dialog.createRadioButton("RadioButton 3", 3,
1,"Tab 2.Group 1")
testBool := testBool AND Dialog.createRadioButton("RadioButton 4", 20,
1,"Tab 2.Group 1")

Dialog.setGroupID("RadioButton 1", 1)-- row 0
Dialog.setGroupID("RadioButton 2", 1)-- row 0
Dialog.setGroupID("RadioButton 3", 2)-- row 1
Dialog.setGroupID("RadioButton 4", 2)-- row 1
-- radio button in different groups
testBool := testBool AND Dialog.createRadioButton("RB 1", 1, 0,"Tab
2.Group 2")
testBool := testBool AND Dialog.createRadioButton("RB 2", 1, 1,"Tab
2.Group 2")

```

```

testBool := testBool AND Dialog.createRadioButton("RB 3", 1, 0, "Tab
2.Group 3")
testBool := testBool AND Dialog.createRadioButton("RB 4", 1, 1, "Tab
2.Group 3")

Dialog.setGroupID("RB 1", 1)
Dialog.setGroupID("RB 2", 1)
Dialog.setGroupID("RB 3", 2)
Dialog.setGroupID("RB 4", 2)

--- Image in the dialog
testBool := testBool AND Dialog.createImage("My_Image", 25, 0, "Tab
1.Group 1")
dialog.setIcon("My_Image", "my icon")
testBool := testBool AND (dialog.getIcon("My_Image")= "my icon")

-- DropDownList
testBool := testBool AND dialog.createDropDownListBox("unit", 14,
1,10, "Tab 1.Group 1")

var units:list[string]
units.create
units.insert(1, "mm")
units.insert(2, "m")
units.insert(3, "km")
testBool := testBool AND dialog.setList("unit", units)
dialog.setCallbackArgument("unit", "DropDownList")

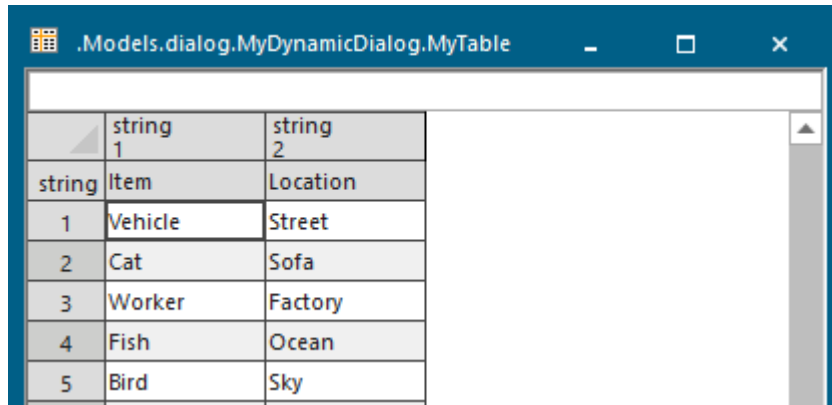
-- List box
testBool := testBool AND dialog.createListBox("ListBox", 17, 1, 18, 2,
"Tab 1.Group 2")
testBool := testBool AND dialog.setList("ListBox", units)
dialog.setCallbackArgument("ListBox", "ListBox Arg")

-- List view
testBool := testBool AND dialog.createListView("List view", 0, 2, 36, 6,
"Tab 3")
testBool := testBool AND dialog.setTable("List view", MyTable)
dialog.setCallbackArgument("List view", "List view Arg")
-- TextBox
testBool := testBool AND dialog.createEditTextBox("Text Box", 0, 2,
13, "Tab 1.Group 2")
testBool := testBool AND dialog.setCaption("Text Box", "Enter Password")
testBool := testBool AND dialog.setPasswordMasking("Text Box", true)

dialog.open
print testBool

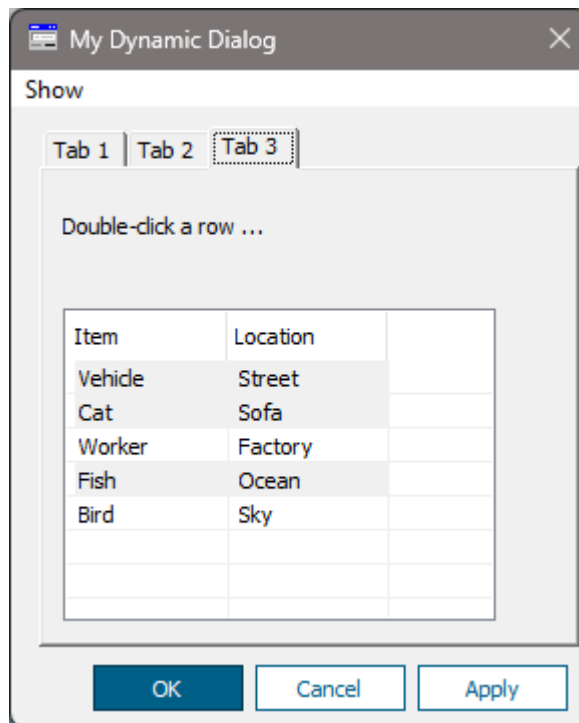
```

- We entered the items, which **Tab 3** is going to show into the *DataTable* named MyTable.



	string 1	string 2
string	Item	Location
1	Vehicle	Street
2	Cat	Sofa
3	Worker	Factory
4	Fish	Ocean
5	Bird	Sky

- We ran the *Method* named *createDialog* to enter what we programmed into the *Dialog* and show it. You'll notice that it now shows the **Name** and the **Label** that we typed into *createDialog* in the *Frame* named *MyDynamicDialog*.



- We programmed the action that double-clicking a row in the table on **Tab 3** executes:

```
param action : string
var row:integer

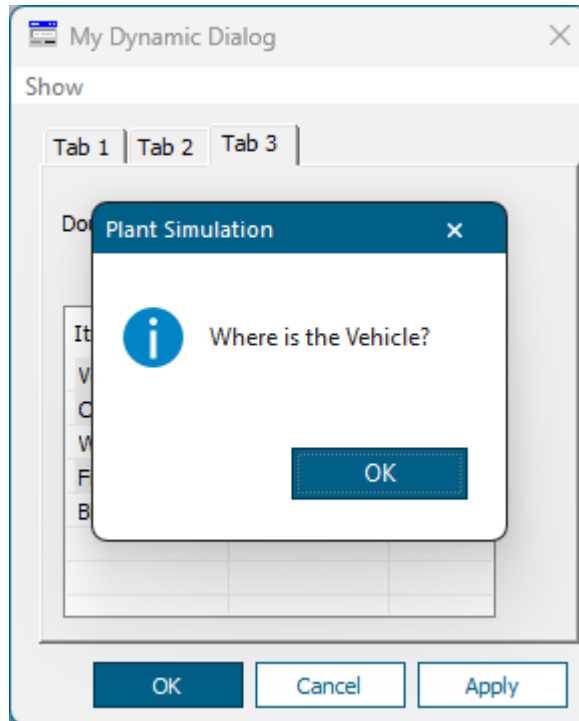
print "Action : ",action
switch action
case "Open"
case "Apply"
case "Close"
case "Button Arg"
    @.setPasswordMasking("Text Box",NOT @.getPasswordMasking("Text Box"))
```

```

case "List view Arg"
    row := @.getIndex("List view")
    promptmessage(to_str("Where is the ", MyTable[1,row],"?"))
case "Check box"
    @.setSensitive("A_Button", @.getCheckBox("Check box"))
end

```

Double-clicking a row in the table shows a message box with the question **Where is the ...?**



Compare [Program Actions which the Dialog Items Execute](#)

Compare [Program Actions for Interacting with the Dialog](#)

Back to [Set Parameters for Objects in Your Own Dialog](#)

Program Actions which the Dialog Items Execute

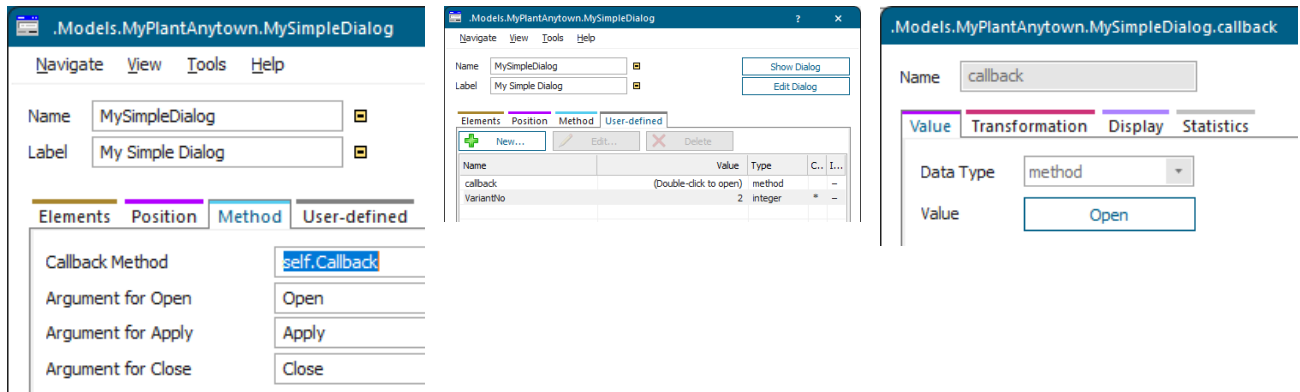
During or after you inserted the dialog items you need, you will have to program the actions, which these dialog items execute, when the user enters or selects a setting. For this you have to enter the respective source code into a *callback method*.

- By default the *callback method* is a *user-defined attribute* of data type *method* named `self.callback`. In most cases you will use that.

To open the default *callback method*, do one of the following:

- On the tab **Method**, click into the text box **Callback Method** and press **F2** or by hold down **Shift** and double-click **Callback**.

- On the tab **User-defined Attributes**, double-click **Callback** and then click **Open** in the dialog **User-defined Attributes**.



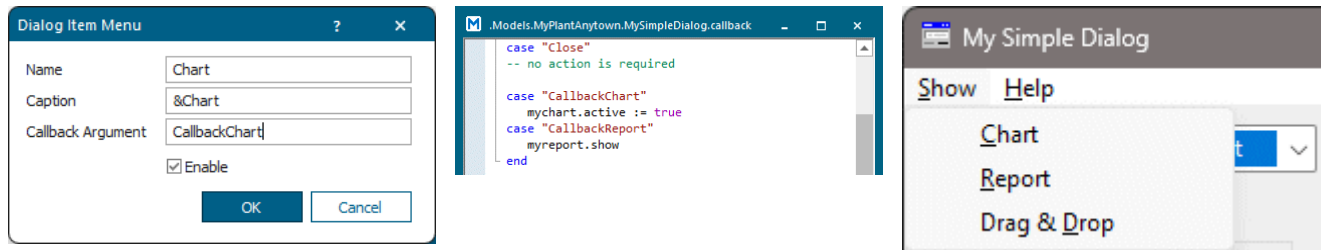
- If your modeling needs warrant it, you can also use a *Method*, which you inserted into a *Frame* or into a folder in the *Class Library*. This will be the case, if several of your dialogs, which do not inherit items, can use the same *callback method* and, for this reason, have to store the method at a single location.

Enter the **Callback Argument** for each of the dialog items into this method and type in the statements (source code) you want it execute. This parameter is the same parameter that you entered into the dialogs of the individual dialog items. Be aware that the parameters are case-sensitive! The *methods* and the *attributes* of the *Dialog* allow you to program callback actions with as much complexity as you need.

The *callback method* executes the callback parameter when the user:

- Closes a drop-down list box.
- Selects and double-clicks an item in a list box.
- Changes the contents of a text box and selects another dialog item afterwards, clicks in another text box, or clicks **OK**, **Apply**, or **Cancel**.
- Clicks a button.
- Selects or clears a check box.
- Selects a radio button.
- Selects a row in a list view and double-clicks it.
- Selects a tab in a tab control.
- Selects a menu or a menu command.

Compare this example:



To open the object *MyChart*, we entered this instruction:

```
case "CallbackChart"
  MyChart.IsShown := true
```

To open the object *MyReport*, we entered this instruction:

```
case "CallbackReport"
  MyReport.show
```

Compare [Program Actions for Interacting with the Dialog](#)

Compare [Design a Simple Dialog](#)

Compare [Design a Tabbed Dialog](#)

Compare [Create a Dialog Dynamically](#)

Compare the [_Methods of the Dialog](#) in the Object Reference

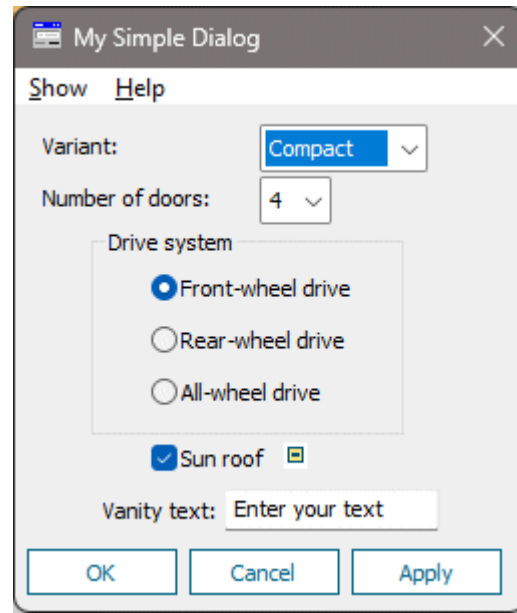
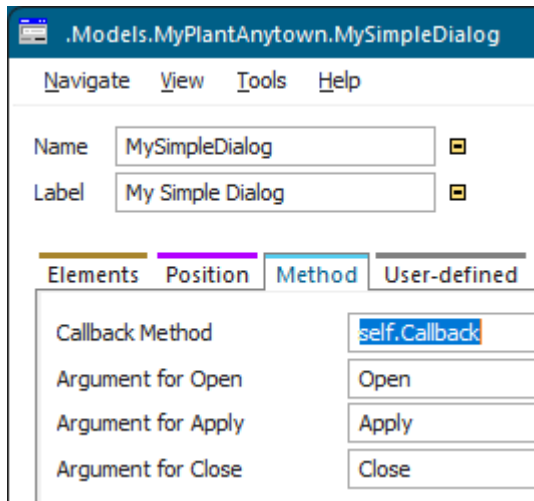
Compare the [_Attributes of the Dialog](#) in the Object Reference

Back to [Set Parameters for Objects in Your Own Dialog](#)

Program Actions for Interacting with the Dialog

After you created your dialog box, you will have to program what happens, when the user opens the dialog box, when he applies settings, and when he closes it.

Enter these actions as callback parameters into the [Callback Method](#).



- The *Dialog* executes the **Open** section of the *callback method* then the user opens the dialog box. It initializes the contents of the dialog box or sets the dialog items to values of your choice.
- The *Dialog* executes the **Apply** section of the *callback method*, when the user clicks **OK** or **Apply** in the *Dialog*. The statements you enter may evaluate new or changed values.

Note

When the user clicks **OK**, the *Dialog* executes the *callback method* twice. The first time it calls the **Apply** section. The second time it calls the **Close** section. When the user clicks **Apply**, the *Dialog* only executes the **Apply** section of the *callback method*.

- The *Dialog* executes the **Close** section of the *callback method*, when the user clicks **Cancel** in the *Dialog* or when he closes it with **Close**  in the title bar.

For our sample dialog box we started defining the settings shown below. When the user opens the dialog, *Plant Simulation* sets the variant of the car to the number of the *user-defined attribute* named *VariantNo*. It selects the check box for the sun roof and it prompts the user to enter vanity text he wants added to his paint job. Naturally, you can preallocate as many values as you want.

The source code looks like this:

```
param action: string
switch action
case "Open"
    @.setIndex("VariantType", @.VariantNo)
    @.setCheckbox("SunRoof", true)
    @.setValue("Vanity text", "Enter your text")
case "Apply"
    @.VariantNo := @.getIndex("VariantType")
case "Close"
```

```
// no action is required

case "CallbackChart"
  MyChart.IsShown := true
case "CallbackReport"
  MyReport.show
end
```

Compare [Design a Simple Dialog](#)

Compare [Design a Tabbed Dialog](#)

Compare [Create a Dialog Dynamically](#)

Compare the [Tab Method](#) of the Dialog

Back to [Set Parameters for Objects in Your Own Dialog](#)

Setting Parameters with the AttributeExplorer

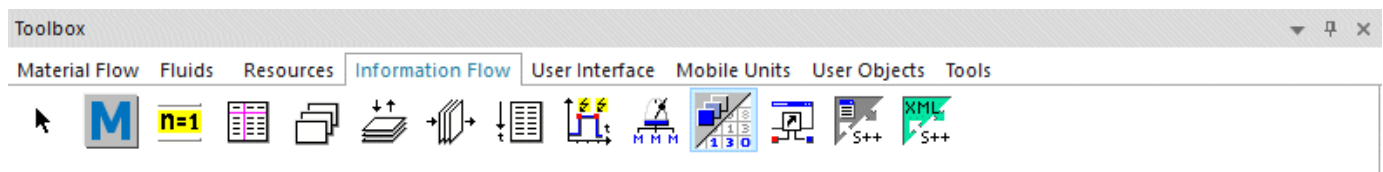
Set Parameters with the AttributeExplorer

Instead of opening the dialog of each and every material flow object in your model and entering values for a single attribute or for a number of attributes into the text boxes, you can define which attributes of which objects the *AttributeExplorer* gets and shows in a list window, when you click **Show Explorer**.

This way you can manage the attributes defining the settings of the individual stations in your simulation model at a single location. You can then enter different values for the capacities, the times, etc. *Plant Simulation* writes these values back to the dialogs of the objects and uses them in your simulation model. You can also export this settings table as a tab-delimited text file and import this file into the *AttributeExplorer* of another simulation model, thus using identical settings in several of your simulation models.

You can also use the *AttributeExplorer* to find objects of the type you define, to find attributes, for example the position of a number of objects, to align them in the *Frame*, etc.

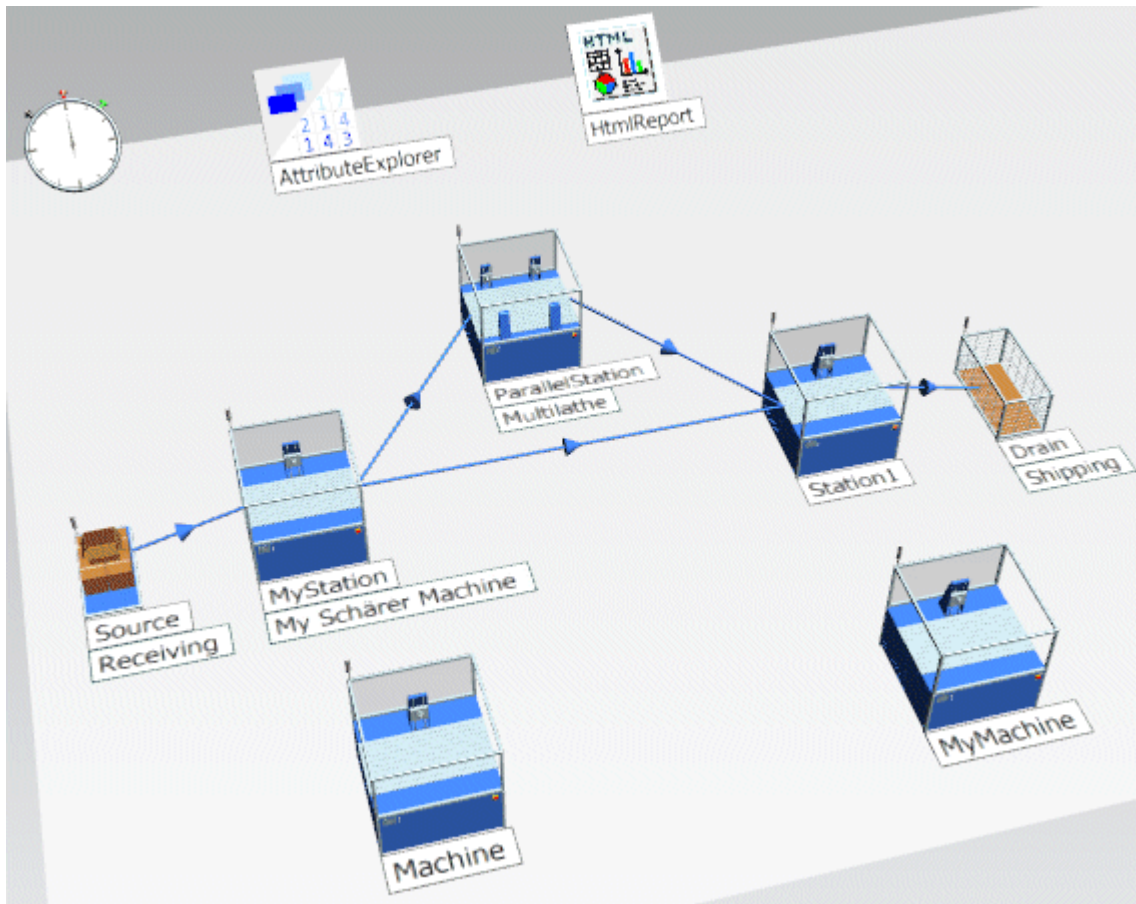
You can insert the **AttributeExplorer**  into your simulation model from the folder **InformationFlow** in the *Class Library* or from the toolbar **Information Flow** in the *Toolbox*.



Depending on what you would like to accomplish, you will:

- Specify the Objects You Want to Parametrize
- Specify the Attributes You Want to View or Change
- Select How to Show the Objects and the Names
- Find Objects and Attributes

Our finished model looks like this:



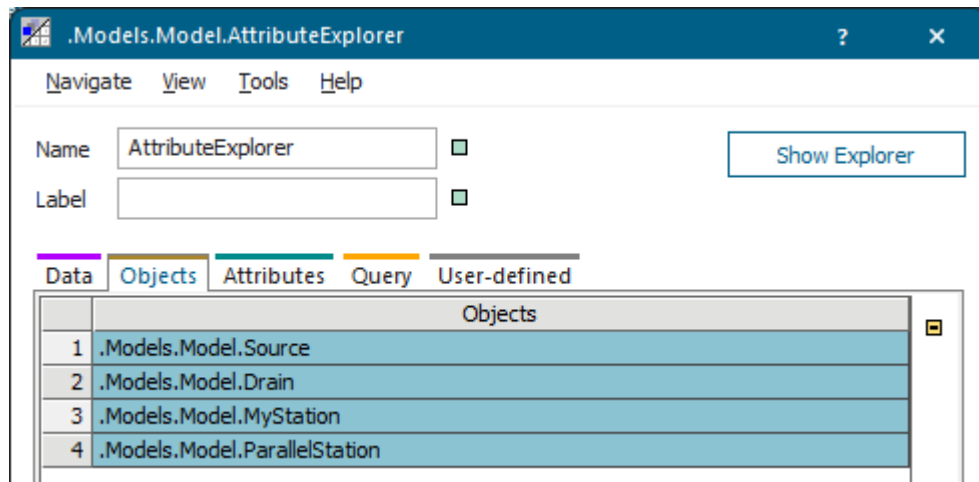
Back to Set Parameters in the Model

Specify the Objects You Want to Parametrize

Before you can type in data, click the **Inheritance** ☐ check box so that it looks like this ☒.

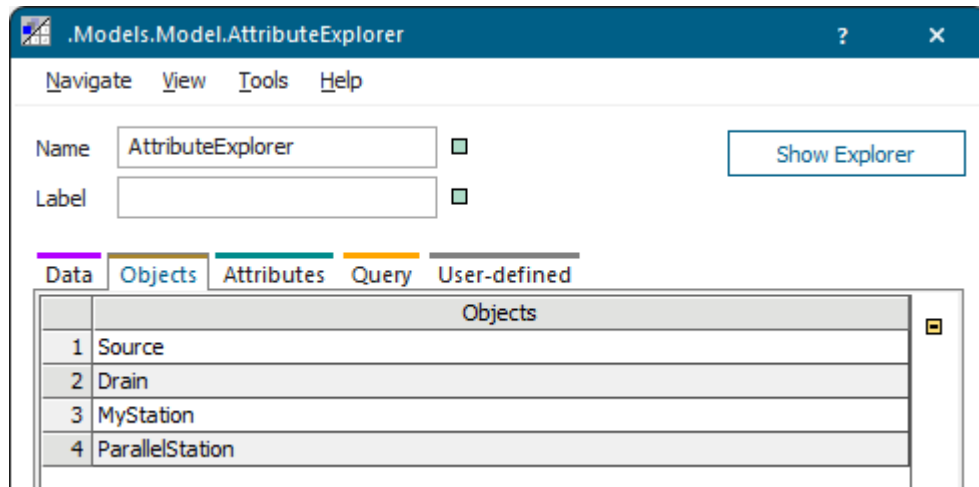
- To view or to edit the attributes of an object, drag that object from the *Frame* window over the tab **Objects** in the open dialog of the *AttributeExplorer* and drop it there. *Plant Simulation* inserts the absolute path to and the name of the object into the selected cell of the list on the tab.

You can also select and then drag-and-drop multiple objects at the same time!



Plant Simulation adds the object in the order in which you selected them in the *Frame*.

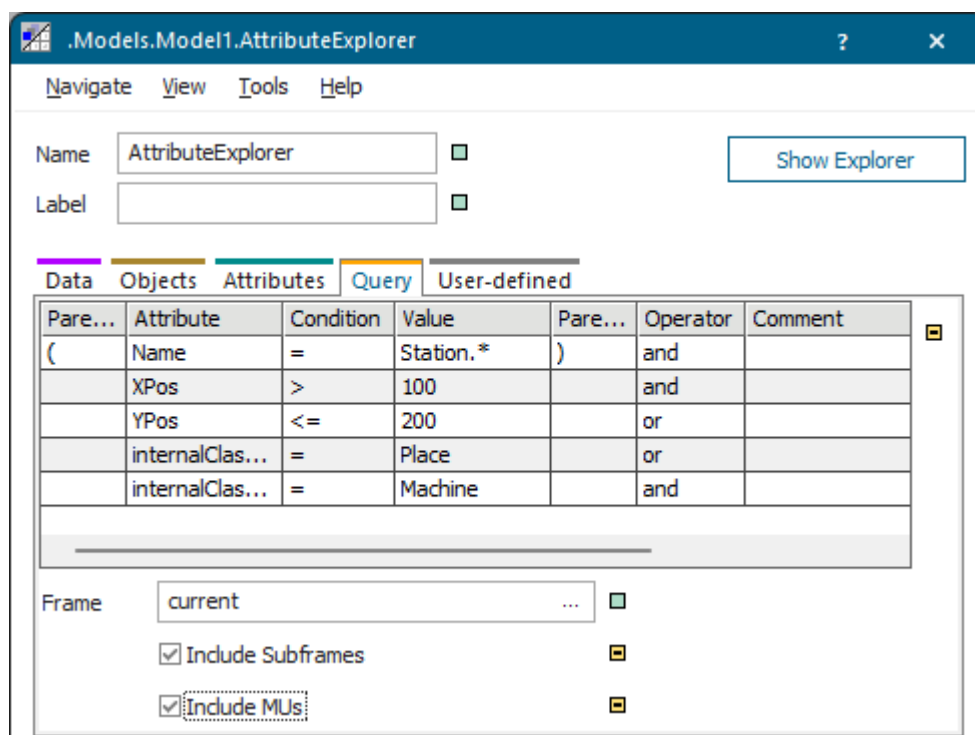
- To just add the name of the object, drag that object from the *Frame* window over the icon of the *AttributeExplorer* and drop it there.



- To view or to edit the attributes of **all objects of one class**, click the tab **Query** and type in **InternalClassType** as the **Attribute** and the actual internal class type of the object as the **Value** of the object there.

Note

For the built-in objects, the read-only attribute **InternalClassType** returns the type of the object.



- To show what you have defined, click **Show Explorer**.

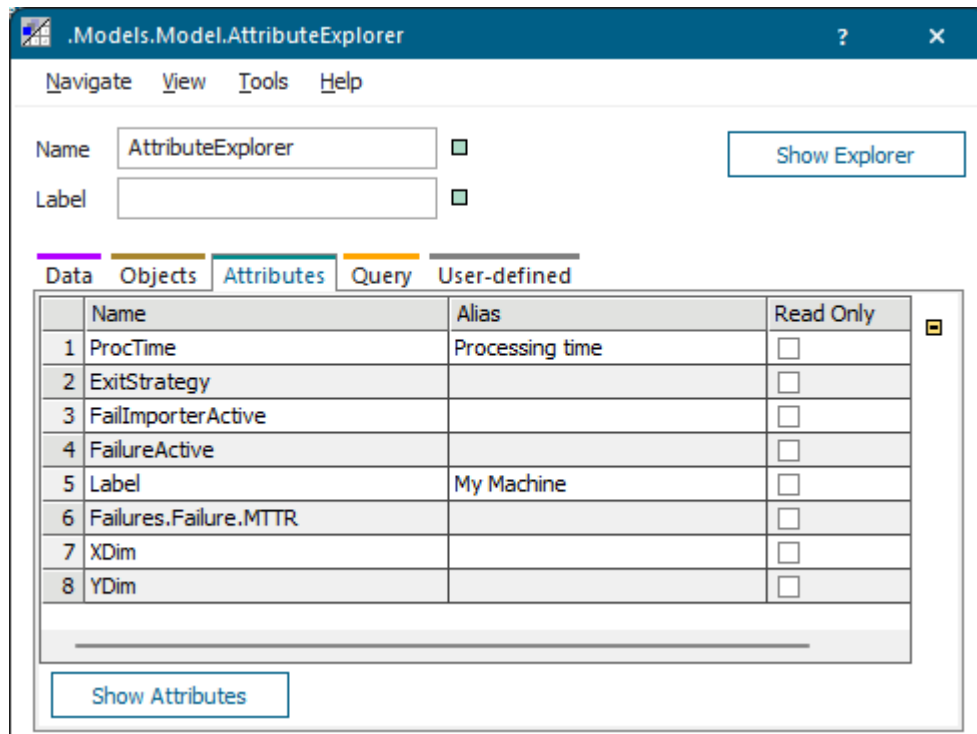
	ProcTime	ExitStrategy	FailImporterActive	FailureActive	Label	Failures.Failure.MTTR	XDim	YDim
Drain	0.0000		false	true	Shipping			
MyStation	1:00.0000	Cyclic	false	true	My Schärer Machine	1:00.0000		
ParallelStation	1:00.0000	Cyclic	false	true	Multilathe		2	2
Source		Cyclic	false	true	Receiving			
Machine	1:00.0000	Cyclic	false	true				
MyMachine	1:00.0000	Cyclic	false	true				
MyStation	1:00.0000	Cyclic	false	true	My Schärer Machine	1:00.0000		
ParallelStation	1:00.0000	Cyclic	false	true	Multilathe		2	2
Source		Cyclic	false	true	Receiving			
Station1	1:00.0000	Cyclic	false	true				

Back to [Set Parameters with the AttributeExplorer](#)

Specify the Attributes You Want to View or Change

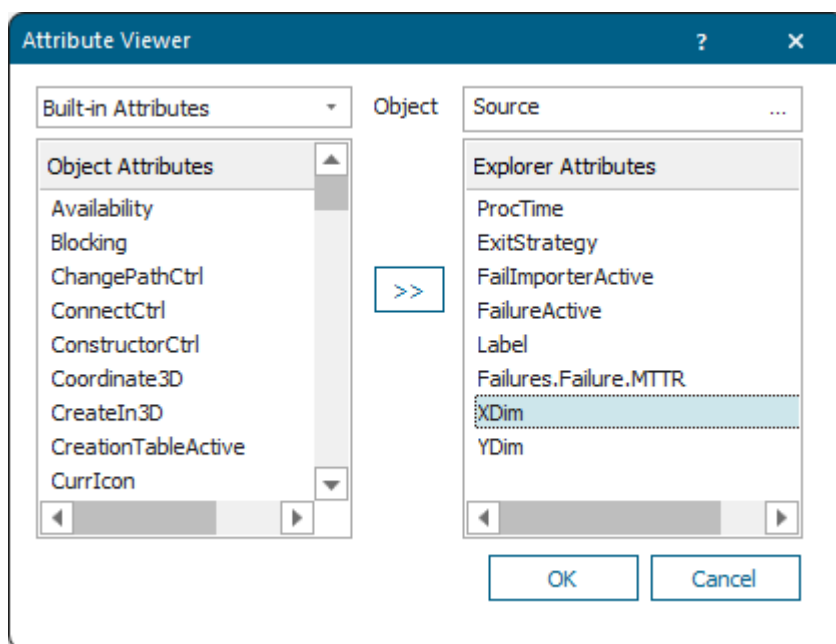
Before you can type in data, click the **Inheritance** check box so that it looks like this .

To type the names of the attributes whose values you want to edit or view into the cells in the column **Name**, click **Show Attributes**.

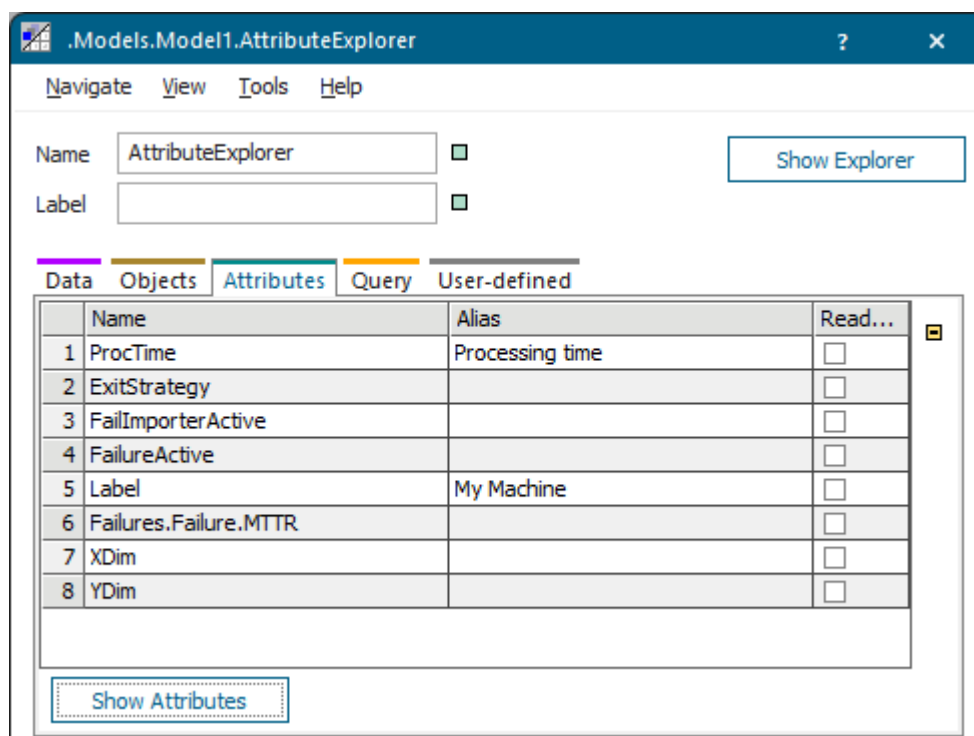


In the dialog **Show Attributes**:

- Click  and select the **Object**, whose attributes you want to show, in the dialog **Select Object**.
- Select from the drop-down list if you want to view the **Built-in Attributes** of this object, or its **User-defined Attributes**, which you, or a co-worker defined. The *AttributeExplorer* then displays these attributes in the list box **Object Attributes**.
- Select a single, or several contiguous attributes (**Shift+click**), and click this  to add these attributes to the attributes to the list box **Explorer Attributes**.
- Click **OK** to add these attributes to the list, which the *AttributeExplorer* displays, on the tab **Attributes**.



If the predefined name of an attribute is not meaningful enough for your purposes, type in a descriptive term for that attribute that suits your modeling situation into the corresponding cell in the column **Alias**.



If you want an attribute to be read only, click into the cell **Read Only**. Then, you can just view, but not edit that value in the list that the button **Show Explorer** opens.

Plant Simulation displays the background of the cells in different colors to show if the attribute is **watchable** or not.

Color	Description
blue	The attribute is not watchable.
white	The attribute is watchable.
gray	You entered a wrong name for a built-in attribute.

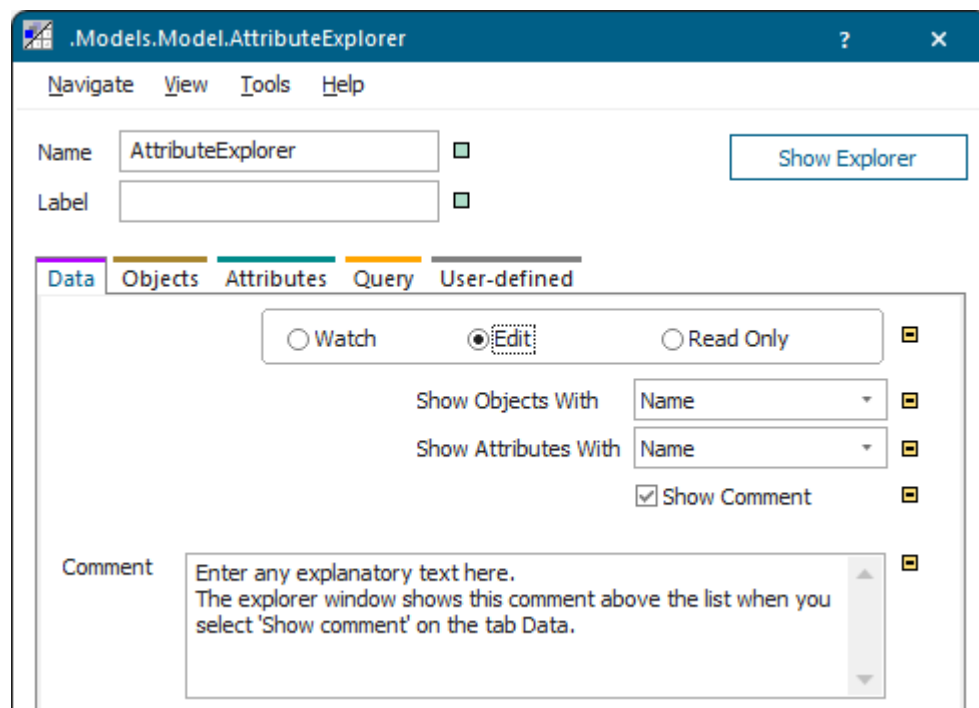
To manipulate the contents of the list, use the commands on the [Context Menu of Embedded Lists](#).

Back to [Set Parameters with the AttributeExplorer](#)

Select How to Show the Objects and the Names

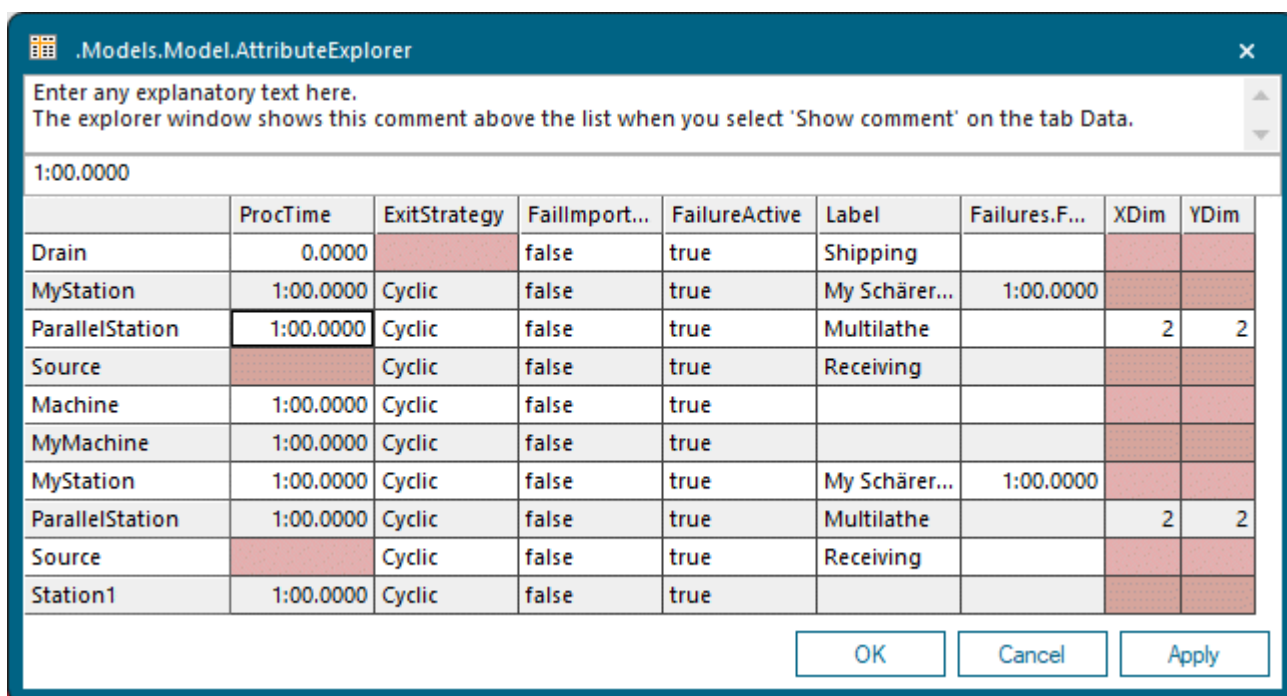
Note

Before you can type in data, click the **Inheritance** check box so that it looks like this .



Select what you want to do with the attributes you added to the *AttributeExplorer*:

- To allow you to edit the values of the attributes of any one of the stations you typed in, select **Edit**  **Edit**. Then, click **Show Explorer**, click in any of the cells that are available, and type in another value. *Plant Simulation* writes the values you changed back to the dialogs of the objects, when you click **Apply/OK**.



- To display the values of the **watchable** attributes of any of the stations you typed in, select **Watch**  **Watch**. Then, click **Show Explorer**. You will notice that you can only view the values, but not edit them. *Plant Simulation* displays the background of the cells in different colors to show if the attribute is **watchable** or not.

Color	Description
blue	The attribute is not watchable.
white	The attribute is watchable.
gray	You entered a wrong name for a built-in attribute.

.Models.Model.AttributeExplorer

Enter any explanatory text here.
The explorer window shows this comment above the list when you select 'Show comment' on the tab Data.

	ProcTime	ExitStrategy	FailImport...	FailureActive	Label	Failures.F...	XDim	YDim
Drain	0.0000		false	true	Shipping			
MyStation	1:00.0000	Cyclic	false	true	My Schärer...	1:00.0000		
ParallelStation	1:00.0000	Cyclic	false	true	Multilathe		2	2
Source		Cyclic	false	true	Receiving			
Machine	1:00.0000	Cyclic	false	true				
MyMachine	1:00.0000	Cyclic	false	true				
MyStation	1:00.0000	Cyclic	false	true	My Schärer...	1:00.0000		
ParallelStation	1:00.0000	Cyclic	false	true	Multilathe		2	2
Source		Cyclic	false	true	Receiving			
Station1	1:00.0000	Cyclic	false	true				

- To just show the values of the attributes of the stations you typed in, select **Read Only** ☒ **Read Only**. Then, click **Show Explorer**. You will notice that you can only view the values, but not edit them.

.Models.Model.AttributeExplorer

Enter any explanatory text here.
The explorer window shows this comment above the list when you select 'Show comment' on the tab Data.

	ProcTime	ExitStrategy	FailImport...	FailureActive	Label	Failures.F...	XDim	YDim
Drain	0.0000		false	true	Shipping			
MyStation	1:00.0000	Cyclic	false	true	My Schärer...	1:00.0000		
ParallelStation	1:00.0000	Cyclic	false	true	Multilathe		2	2
Source		Cyclic	false	true	Receiving			
Machine	1:00.0000	Cyclic	false	true				
MyMachine	1:00.0000	Cyclic	false	true				
MyStation	1:00.0000	Cyclic	false	true	My Schärer...	1:00.0000		
ParallelStation	1:00.0000	Cyclic	false	true	Multilathe		2	2
Source		Cyclic	false	true	Receiving			
Station1	1:00.0000	Cyclic	false	true				

- Select how you want to **show the objects** in the *Explorer* window in the leftmost column:
 - With their entire **Path**:

.Models.Model.AttributeExplorer	
Enter any explanatory text here. The explorer window shows this comment above	
	ProcTime
Drain	0.0000
MyStation	1:00.0000
ParallelStation	1:00.0000
Source	
*.Models.Model.Machine	1:00.0000
*.Models.Model.MyMachine	1:00.0000
*.Models.Model.MyStation	1:00.0000
*.Models.Model.ParallelStation	1:00.0000
*.Models.Model.Source	
*.Models.Model.Station1	1:00.0000

Note

We dragged the *Source* from the *Frame* window over the tab **Data** in the open dialog of the *AttributeExplorer* and dropped it there. This inserted its absolute path. We dragged the *Drain* from the *Frame* window over the icon of the *AttributeExplorer* and dropped it there. This inserted its name only.

- With their **Name** only:

.Models.Model.AttributeExplorer	
Enter any explanatory text here. The explorer window shows this comment above	
	ProcTime
Drain	0.0000
MyStation	1:00.0000
ParallelStation	1:00.0000
Source	
Machine	1:00.0000
MyMachine	1:00.0000
MyStation	1:00.0000
ParallelStation	1:00.0000
Source	
Station1	1:00.0000

- With their **Label** only:

 .Models.Model.AttributeExplorer

Enter any explanatory text here.
The explorer window shows this comment above

	ProcTime
Multilathe	0.0000
My Schärer Machine	1:00.0000
Receiving	1:00.0000
Shipping	
	1:00.0000
	1:00.0000
	1:00.0000
Multilathe	1:00.0000
My Schärer Machine	
Receiving	1:00.0000

- Select how you want to **show the attributes** in the *Explorer* window:
 - ◊ With their **Name**.

 .Models.Model.AttributeExplorer

Enter any explanatory text here.
The explorer window shows this comment above the list when you select 'Show comment' on the tab Data.

	ProcTime	ExitStrategy	FailImporterActive	FailureActive	Label
Drain	0.0000		false	true	Shipping
MyStation	1:00.0000	Cyclic	false	true	My Schärer Machine
ParallelStation	1:00.0000	Cyclic	false	true	Multilathe
Source		Cyclic	false	true	Receiving
Machine	1:00.0000	Cyclic	false	true	
MyMachine	1:00.0000	Cyclic	false	true	
MyStation	1:00.0000	Cyclic	false	true	My Schärer Machine
ParallelStation	1:00.0000	Cyclic	false	true	Multilathe
Source		Cyclic	false	true	Receiving
Station1	1:00.0000	Cyclic	false	true	

- ◊ With their **Alias**. Enter the alias on the tab **Attributes**.

.Models.Model.AttributeExplorer

Enter any explanatory text here.
The explorer window shows this comment above the list when you select 'Show comment' on the tab Data.

	Processing time	ExitStrategy	FaillImporterActive	FailureActive	My Machine
Drain	0.0000		false	true	Shipping
MyStation	1:00.0000	Cyclic	false	true	My Schärer Machine
ParallelStation	1:00.0000	Cyclic	false	true	Multilathe
Source		Cyclic	false	true	Receiving
Machine	1:00.0000	Cyclic	false	true	
MyMachine	1:00.0000	Cyclic	false	true	
MyStation	1:00.0000	Cyclic	false	true	My Schärer Machine
ParallelStation	1:00.0000	Cyclic	false	true	Multilathe
Source		Cyclic	false	true	Receiving
Station1	1:00.0000	Cyclic	false	true	

- If you want to, you can enter an explanation for the objects and values you defined into the text box **Comment**. To create a line break, press **Shift+Enter**.

Data | Objects | Attributes | Query | User-defined

☐ Watch ☐ Edit ☒ Read Only

Show Objects With:

Show Attributes With:

☒ Show Comment

Comment: Enter any explanatory text here.
The explorer window shows this comment above the list when you select 'Show comment.'

To display the text you entered above the list field in the *Explorer* window, select **Show comment** and click **Apply**.

.Models.Model.AttributeExplorer

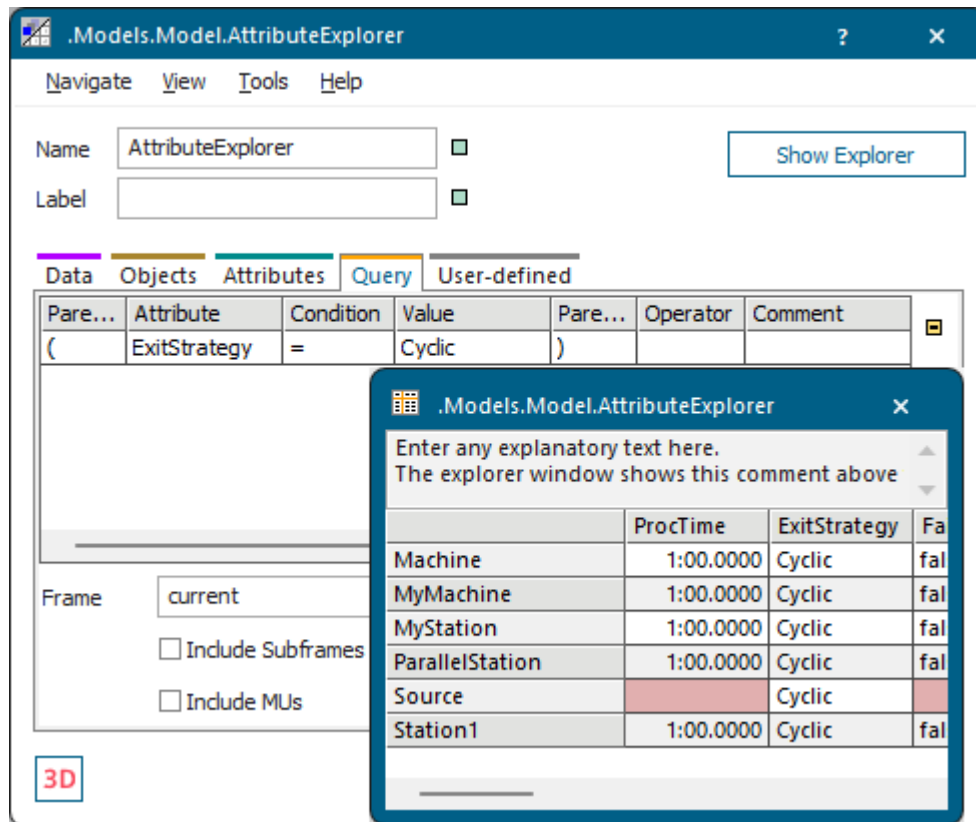
Enter any explanatory text here.
The explorer window shows this comment above the list when you select 'Show comment' on the tab Data.

	ProcTime	ExitStrategy	FaillImporterActive	FailureActive	Label	Failures.F...	XDim	YDim
Drain	0.0000		false	true	Shipping			

Back to [Set Parameters with the AttributeExplorer](#)

Find Objects and Attributes

You can also use the *AttributeExplorer* to find objects in your simulation model. First, enter the criteria into the cells of the list on the tab **Query**. Then, click **Show Explorer**. The *Explorer* window only shows the objects, which meet the search criteria you entered.



If you also want to change the attributes, enter their names on the tab **Attributes**. Then, click **Show Explorer**. Next to the objects the *Explorer* window now also shows the attribute, which you can also change, when you selected **Edit** on the tab **Data**.

You can:

- Select the number of opening **Parentheses** of the query.
- Enter the name of any of the attributes of the object, which the window **Show Attributes and Methods** displays.
- Click in the cell once with the left mouse button and select a **Condition** from the drop-down list. The value of the attribute can be:
 - < (less than)
 - <= (less than or equal to)
 - > (greater than)

- **>=** (greater than or equal to)
- **=** (equal to), compares attributes of data type *real*, *length*, *height*, *speed*, and *time* for the exact value
- **~=** (equal to, case-insensitive) compares if strings are equal not considering lower and upper casing, or compares if the values of attributes are about equal
- **/=** (not equal to)
- **Expr**, which is a regular expression, compare [Show Structure](#). You might, for example, enter **^Inf**, which finds any word beginning with **Inf**.
- **Exists**, checks if the object has the designated attribute or not.

Then enter the value into the corresponding cell in the column **Value**.

- Enter the **Value** of the attribute which the *AttributeExplorer* is to find.
- Select the number of closing **Parentheses** of the query.
- Click in the cell once with the left mouse button and select a boolean **Operator** from the drop-down list. This operator connects the boolean values of the active row with the next row. The boolean values result from evaluating the logical expression you defined in a row of the query table.
 - **and**
 - **or**
- Enter a **Comment**.
- Select the *Frame* in the dialog **Select Object** where you want to start finding the attributes of the objects.

To also include any *Frames* that are located within the *Frame* you selected, select the check box [Include Subframes](#).

Note

Select to show the objects with their path on the tab **Data** for this setting.

Compare [Find Objects and Text in Your Simulation Model](#)

Back to [Set Parameters with the AttributeExplorer](#)